

Data re uploading classifier

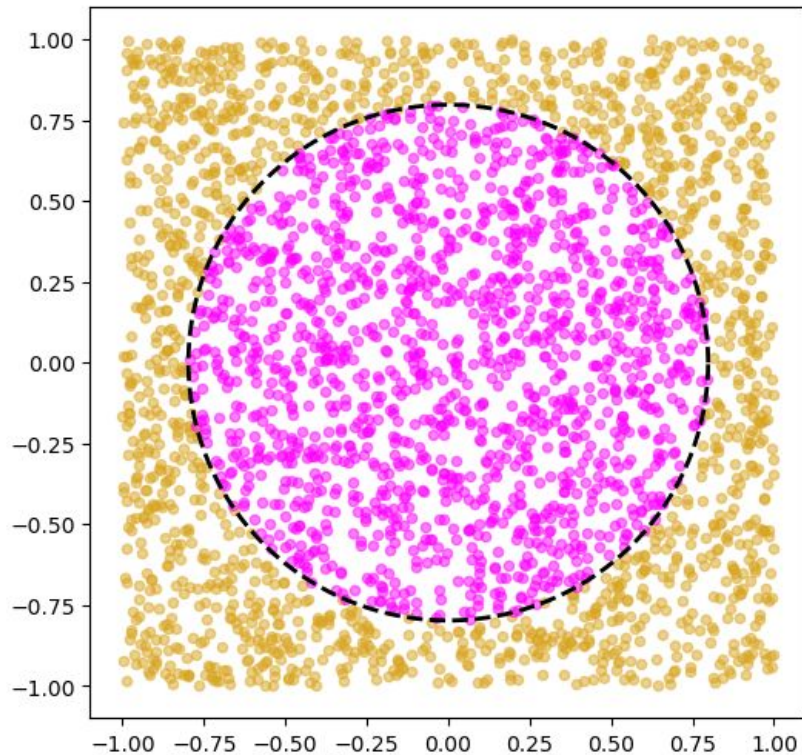
Classic vs quantum algorithms

Synthetic Datasets

Circle dataset:

- Generated with 4200 data points
- For the training, we split 200 data points and 4000 for testing
- 2 Labels: 0 for inside the circle; 1 for outside the circle

```
def circle(samples):  
    #np.random.seed(random_seed)  
    centers = np.array([[0, 0]])  
    radii = np.array([np.sqrt(2/np.pi)])  
    data=[]  
    dim = 2  
    for i in range(samples):  
        x = 2 * (np.random.rand(dim)) - 1  
        #random.rand = create an array of  
        y = 0  
        for c, r in zip(centers, radii):  
            if np.linalg.norm(x - c) < r:  
                y = 1  
        data.append([x, y])  
  
    return data, (centers, radii)
```

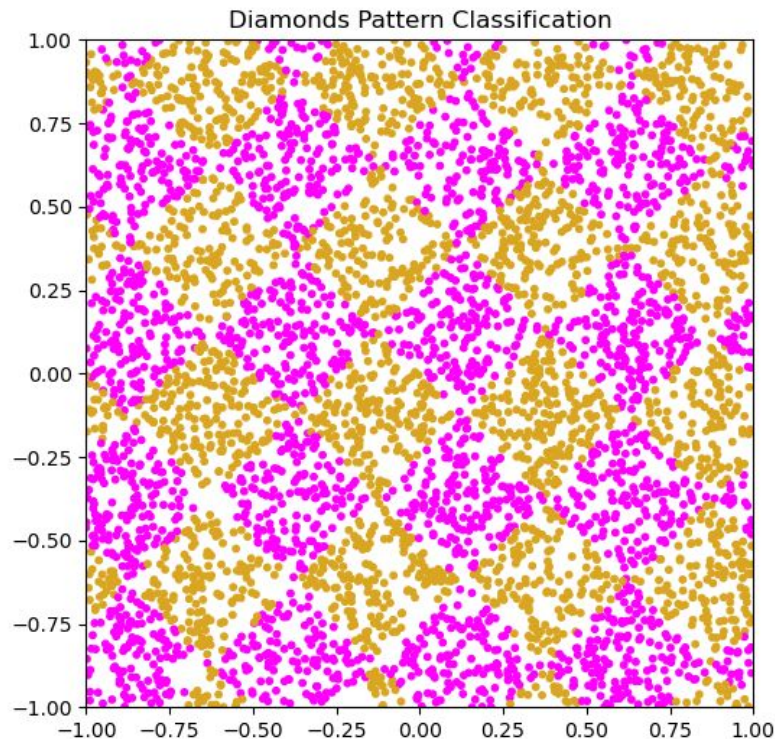


Synthetic Datasets

Diamond pattern dataset:

- Generated with 4200 data points
- For the training, we split 200 data points and 4000 for testing
- 2 Labels: 0 for Fuchsia; 1 for Golden

```
def diamond(samples, random_seed=42):  
    np.random.seed(random_seed)  
    data = []  
    limit = 1  
    scale = 0.5  
  
    x_i = np.random.uniform(-1, limit, samples) #generates the data points inside the square limit  
    y_i = np.random.uniform(-1, limit, samples)  
    x = np.column_stack([x_i, y_i])  
  
    for i in range(len(x)):  
        logic = (np.floor((x[i][0] + x[i][1]) / scale).astype(int) + np.floor((x[i][0] - x[i][1] + scale/2) / scale).astype(int)) % 2  
        data.append((x[i], int(logic)))  
  
    # (x + y) and (x - y) to create diagonals  
    # Default function: (floor((x+y)/s) + floor((x-y+s/2)/s)) mod 2  
    # scale determines the frequency of pattern - if bigger, diamonds get bigger; otherwise, denser  
    return data, (limit, scale)
```

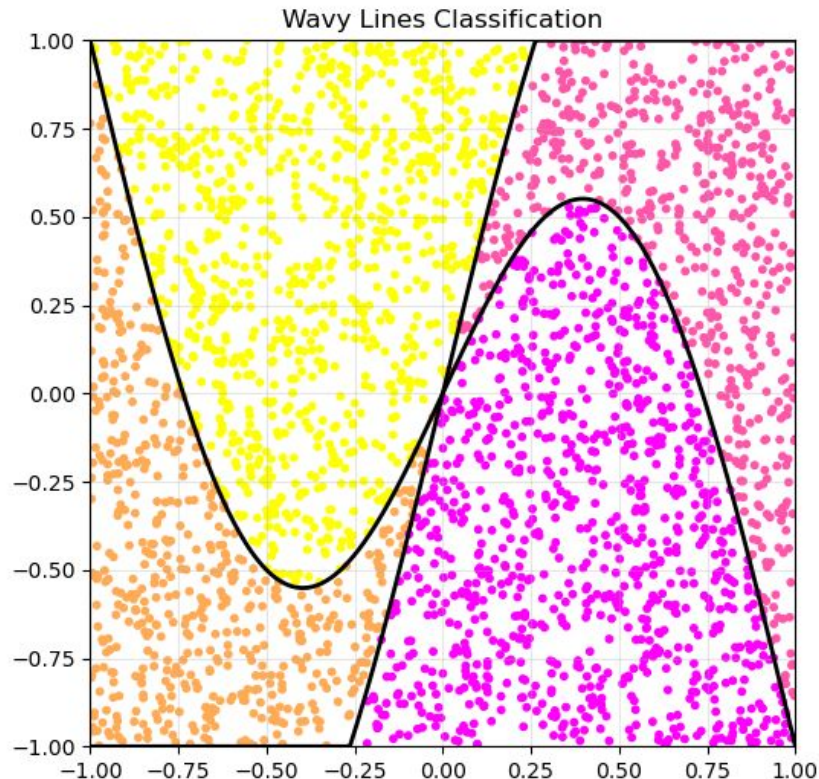


Synthetic Datasets

Wavy lines dataset:

- Generated with 4200 data points
- For the training, we split 200 data points and 4000 for testing
- 4 Labels: 0 <- Below both curves
- 1 <- Below f1 and above f2
- 2 <- Above f1 and below f2
- 3 <- Above both curves

```
def wavy_lines(samples, freq = 1):  
    def fun1(s):  
        return s + np.sin(freq * np.pi * s)  
  
    def fun2(s):  
        return -s + np.sin(freq * np.pi * s)  
  
    data=[]  
    dim=2  
    for i in range(samples):  
        x = 2 * (np.random.rand(dim)) - 1  
        if x[1] < fun1(x[0]) and x[1] < fun2(x[0]): y = 0  
        if x[1] < fun1(x[0]) and x[1] > fun2(x[0]): y = 1  
        if x[1] > fun1(x[0]) and x[1] < fun2(x[0]): y = 2  
        if x[1] > fun1(x[0]) and x[1] > fun2(x[0]): y = 3  
        data.append([x, y])  
  
    return data, freq
```



Iris dataset

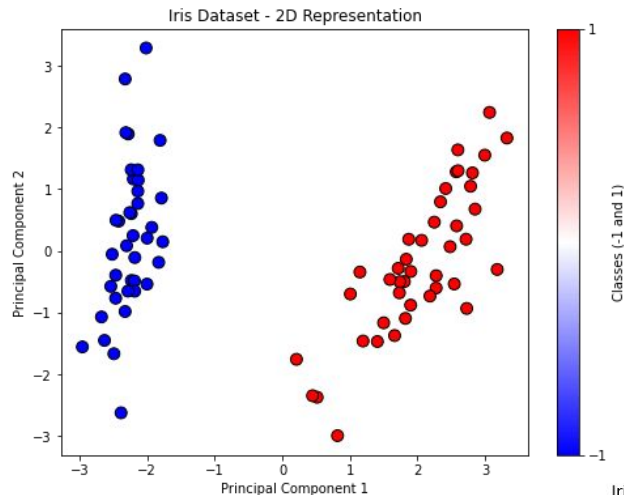
- Dataset obtained from Kaggle - to test the performance of the algorithms in a real life dataset
- We used the “Setosa vs Versicolor” data
- Considering 100 data points

```
iris = load_iris()
X = iris.data
y = iris.target

#Binary Classification (0:setosa,1:versicolor,2:virginica)
if task_type == "setosa_vs_versicolor":
    mask = (y == 0) | (y == 1)
elif task_type == "versicolor_vs_virginica":
    mask = (y == 1) | (y == 2)
else:
    raise ValueError("Please provide a valid task_type.")

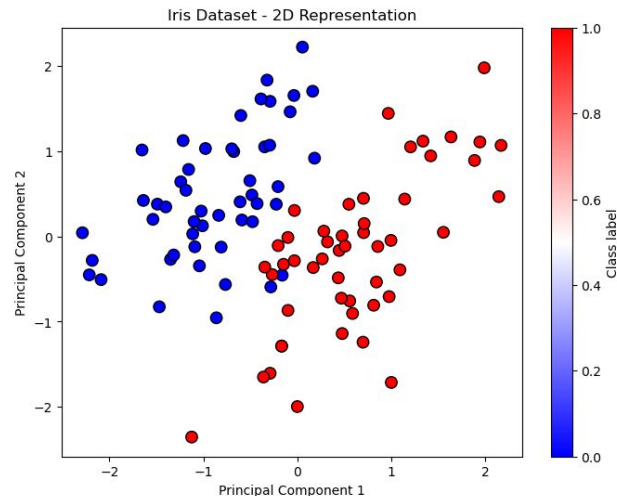
X = X[mask] #data matrices
y = y[mask] #answersheet

#Relabel to -1 and 1 (For quantum expectation values)
if task_type == "setosa_vs_versicolor":
    y = np.where(y == 0, -1, 1) # 0 becomes -1, 1 becomes 1
elif task_type == "versicolor_vs_virginica":
    y = np.where(y == 1, -1, 1) # 1 becomes -1, 2 becomes 1
```



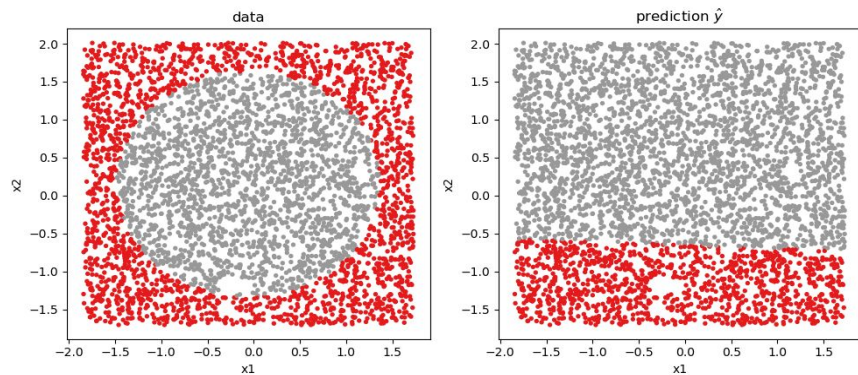
Setosa vs
versicolor

Versicolor
vs virginica

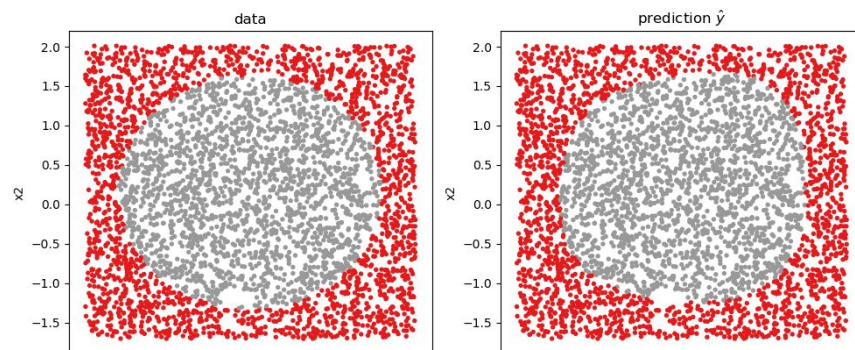


Classic results - Circle

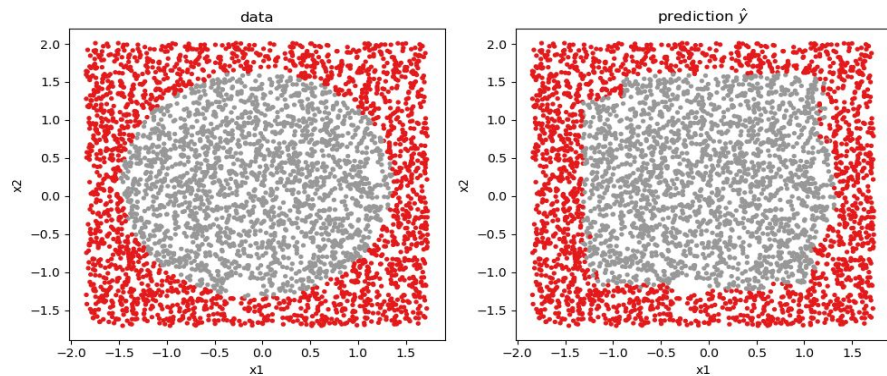
Circle — Logistic Regression



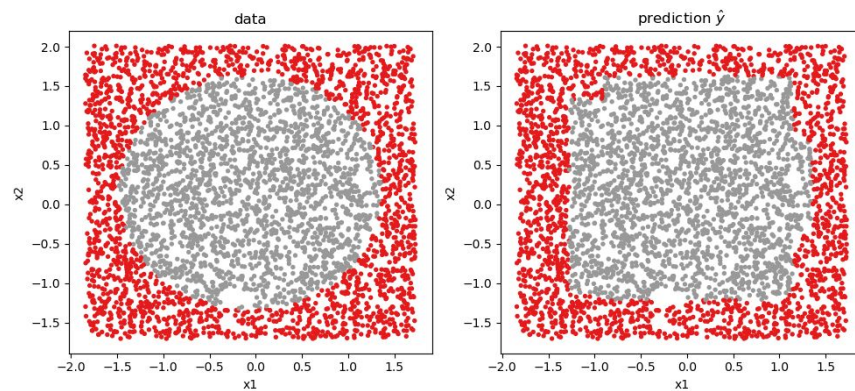
Circle — SVM



Circle — Random Forest



Circle — XGB



--- Logistic Regression (Circle) ---

Accuracy: 0.4410

Classification Report:

	precision	recall	f1-score	support
0	0.45	0.60	0.52	1983
1	0.42	0.28	0.34	2017
accuracy			0.44	4000
macro avg	0.44	0.44	0.43	4000
weighted avg	0.44	0.44	0.43	4000

--- SVM (Circle) ---

Accuracy: 0.9665

Classification Report:

	precision	recall	f1-score	support
0	0.94	1.00	0.97	1983
1	1.00	0.93	0.97	2017
accuracy			0.97	4000
macro avg	0.97	0.97	0.97	4000
weighted avg	0.97	0.97	0.97	4000

--- Random Forest (Circle) ---

Accuracy: 0.9327

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.91	0.93	1983
1	0.91	0.96	0.93	2017
accuracy			0.93	4000
macro avg	0.93	0.93	0.93	4000
weighted avg	0.93	0.93	0.93	4000

--- XGB (Circle) ---

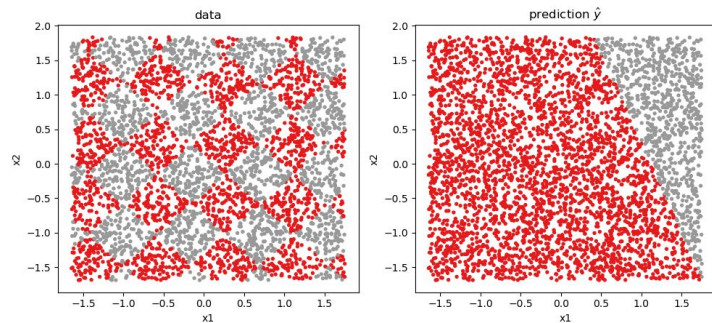
Accuracy: 0.9413

Classification Report:

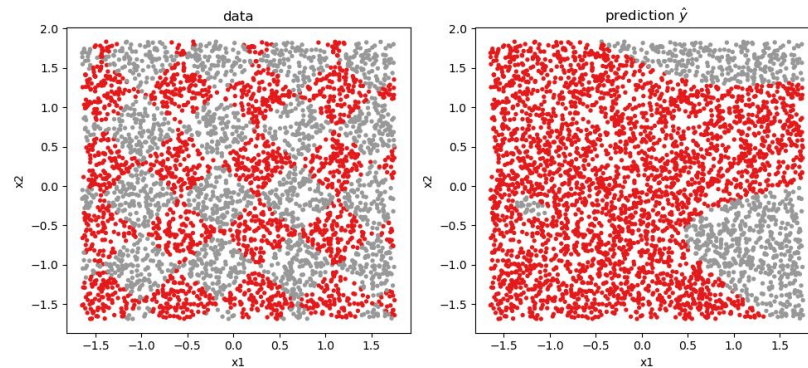
	precision	recall	f1-score	support
0	0.97	0.91	0.94	1983
1	0.92	0.97	0.94	2017
accuracy			0.94	4000
macro avg	0.94	0.94	0.94	4000
weighted avg	0.94	0.94	0.94	4000

Classic results - Diamond

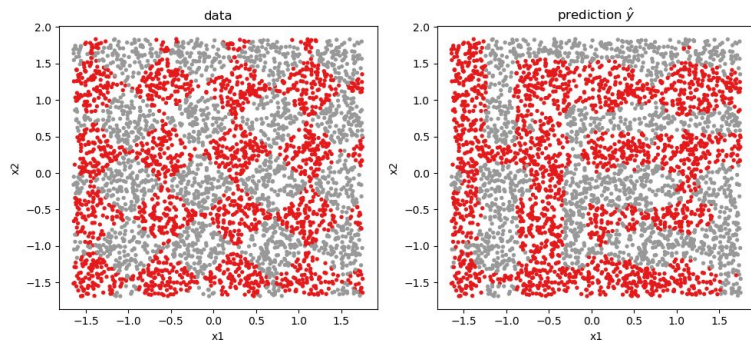
Diamond — Logistic Regression



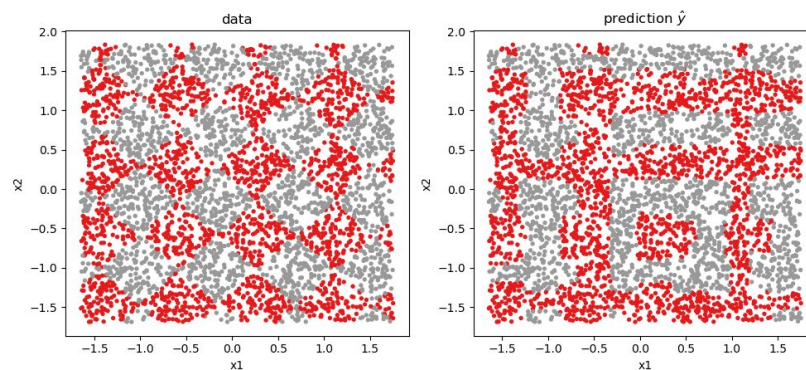
Diamond — SVM



Diamond — Random Forest



Diamond — XGB



--- Logistic Regression (Diamond) ---

Accuracy: 0.5465

Classification Report:

	precision	recall	f1-score	support
0	0.53	0.85	0.65	2014
1	0.61	0.24	0.34	1986
accuracy			0.55	4000
macro avg	0.57	0.54	0.50	4000
weighted avg	0.57	0.55	0.50	4000

--- SVM (Diamond) ---

Accuracy: 0.5597

Classification Report:

	precision	recall	f1-score	support
0	0.54	0.85	0.66	2014
1	0.63	0.27	0.38	1986
accuracy			0.56	4000
macro avg	0.59	0.56	0.52	4000
weighted avg	0.59	0.56	0.52	4000

--- Random Forest (Diamond) ---

Accuracy: 0.7865

Classification Report:

	precision	recall	f1-score	support
0	0.76	0.83	0.80	2014
1	0.81	0.74	0.77	1986
accuracy			0.79	4000
macro avg	0.79	0.79	0.79	4000
weighted avg	0.79	0.79	0.79	4000

--- XGB (Diamond) ---

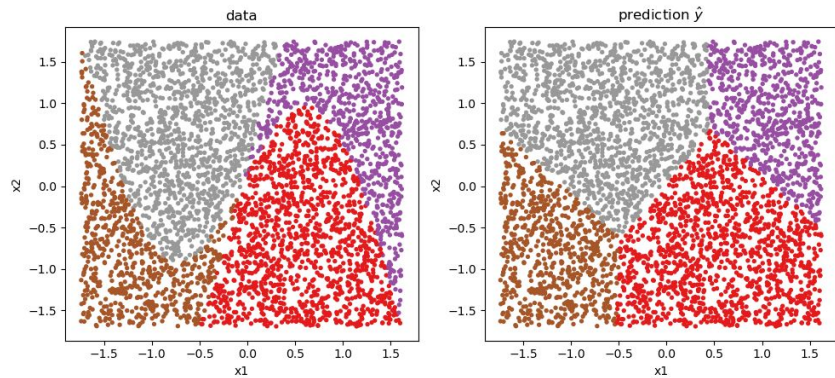
Accuracy: 0.8113

Classification Report:

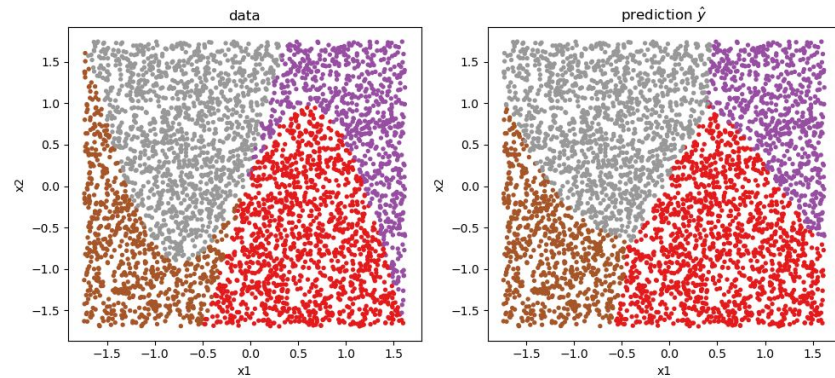
	precision	recall	f1-score	support
0	0.79	0.84	0.82	2014
1	0.83	0.78	0.80	1986
accuracy			0.81	4000
macro avg	0.81	0.81	0.81	4000
weighted avg	0.81	0.81	0.81	4000

Classic results - Wavy lines

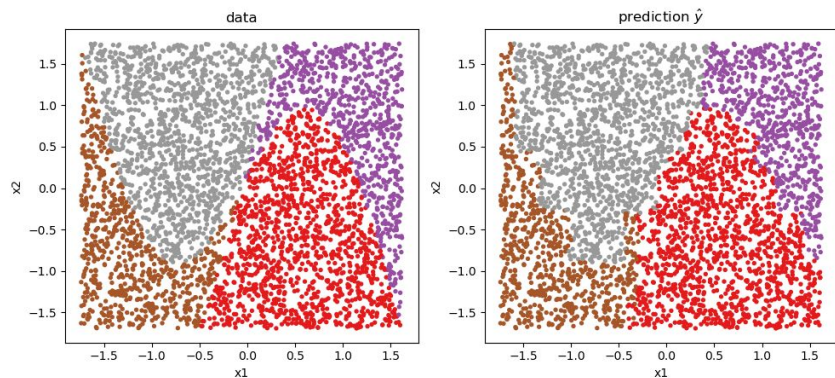
Wavy Lines — Logistic Regression



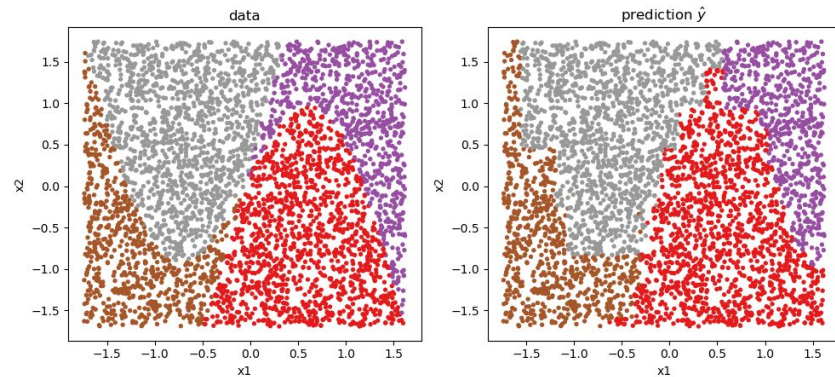
Wavy Lines — SVM



Wavy Lines — Random Forest



Wavy Lines — XGB



--- Logistic Regression (Wavy Lines) ---

Accuracy: 0.8552

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.89	0.88	1283
1	0.85	0.79	0.82	734
2	0.81	0.81	0.81	713
3	0.86	0.88	0.87	1270
accuracy			0.86	4000
macro avg	0.85	0.84	0.85	4000
weighted avg	0.86	0.86	0.85	4000

--- Random Forest (Wavy Lines) ---

Accuracy: 0.9297

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.96	0.95	1283
1	0.95	0.85	0.90	734
2	0.92	0.90	0.91	713
3	0.92	0.96	0.94	1270
accuracy			0.93	4000
macro avg	0.93	0.92	0.92	4000
weighted avg	0.93	0.93	0.93	4000

--- SVM (Wavy Lines) ---

Accuracy: 0.9020

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.96	0.93	1283
1	0.92	0.82	0.87	734
2	0.87	0.84	0.85	713
3	0.91	0.93	0.92	1270
accuracy			0.90	4000
macro avg	0.90	0.89	0.89	4000
weighted avg	0.90	0.90	0.90	4000

--- XGB (Wavy Lines) ---

Accuracy: 0.9203

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.97	0.94	1283
1	0.95	0.79	0.87	734
2	0.91	0.92	0.91	713
3	0.93	0.94	0.94	1270
accuracy			0.92	4000
macro avg	0.92	0.91	0.91	4000
weighted avg	0.92	0.92	0.92	4000

Classic results - Iris setosa vs. versicolor

Logistic Regression Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Random Forest Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

SVM Accuracy: 1.0

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

XGB Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

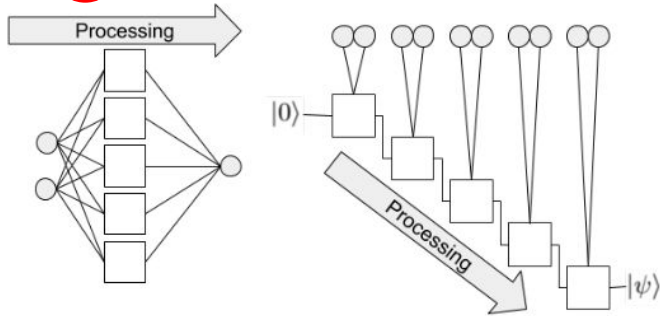
Classical results - Iris Versicolor vs. Virginica

Logistic Regression Accuracy: 0.8					SVM Accuracy: 0.9				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	0.90	0.75	0.82	12	0	1.00	0.83	0.91	12
1	0.70	0.88	0.78	8	1	0.80	1.00	0.89	8
					...				
accuracy			0.80	20	accuracy			0.95	20
macro avg	0.80	0.81	0.80	20	macro avg	0.94	0.96	0.95	20
weighted avg	0.82	0.80	0.80	20	weighted avg	0.96	0.95	0.95	20
Random Forest Accuracy: 0.95					XGB Accuracy: 0.95				
	precision	recall	f1-score	support		precision	recall	f1-score	support
0	1.00	0.92	0.96	12	0	1.00	0.92	0.96	12
1	0.89	1.00	0.94	8	1	0.89	1.00	0.94	8
accuracy			0.95	20	accuracy			0.95	20
macro avg	0.94	0.96	0.95	20	macro avg	0.94	0.96	0.95	20
weighted avg	0.96	0.95	0.95	20	weighted avg	0.96	0.95	0.95	20

Single - qubit

The global structure of any quantum circuit can be divided into three elements: uploading of information onto a quantum state, processing of the quantum state, and measurement of the final state. It is

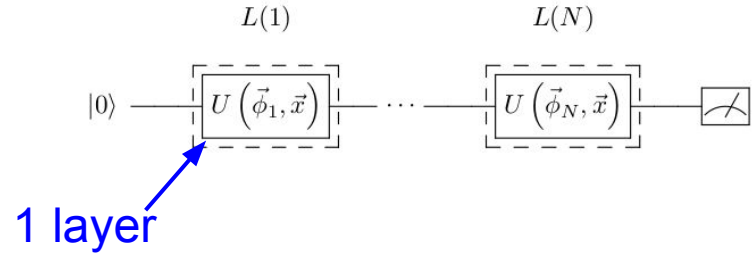
① Re-uploading process



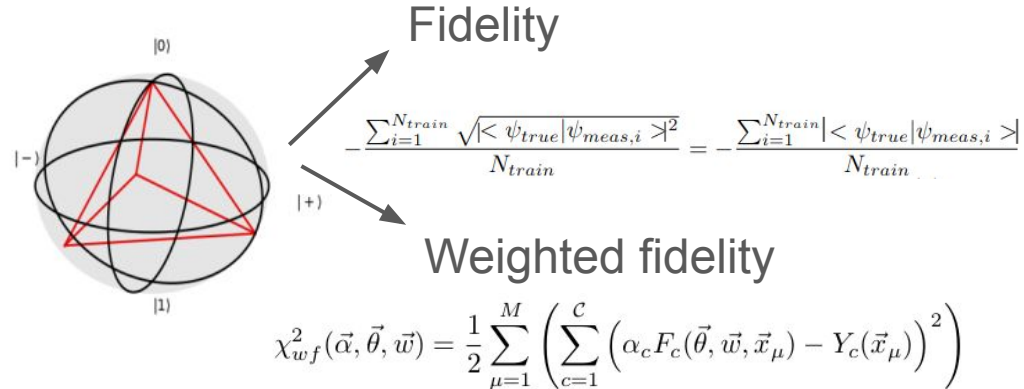
(a) Neural network

(b) Quantum classifier

② Rotation



③ Measurement and classification



1 Data re-uploading and 2 Rotation

```
def single_qubit_circuit(theta_aux):
    qc = QuantumCircuit(1)

    for i in range(theta_aux.shape[1]):
        mat = U3(theta_aux[0, i, :])
        qc.append(UnitaryGate(mat), [0])

    return qc
```

```
def U3(theta3):
    t, phi, lam = theta3
    c = np.cos(t / 2)
    s = np.sin(t / 2)
    e_phi = np.exp(1j * phi / 2)
    e_lambda = np.exp(1j * lam / 2)

    return np.array([
        [ c * e_phi * e_lambda,          -s * e_phi * np.conj(e_lambda)],
        [ s * np.conj(e_phi) * e_lambda,  c * np.conj(e_phi) * np.conj(e_lambda)]
    ])
```

$$U(\vec{\phi}) = U(\phi_1, \phi_2, \phi_3) = e^{i\phi_2\sigma_z} e^{i\phi_1\sigma_y} e^{i\phi_3\sigma_z},$$

$$\vec{\phi}(\vec{x}) = (\phi_1(\vec{x}), \phi_2(\vec{x}), \phi_3(\vec{x})) = \vec{\theta} + \vec{w} \circ \vec{x}.$$

$$\mathfrak{D}_{\text{aux}} = \begin{pmatrix} \theta_{1,1} + w_{1,1}X_1 & \theta_{1,2} + w_{1,2}X_2 & \theta_{1,3} + w_{1,3}X_3 \\ \dots & \dots & \dots \\ \theta_{n,1} + w_{n,1}X_1 & \theta_{n,2} + w_{n,2}X_2 & \theta_{n,3} + w_{n,3}X_3 \end{pmatrix}$$

Where:

$\theta_{i,j}$ i = layer
 j = dimension (up to 3)

3 Measurement and classification

```

if n_classes == 2:
    reprs[0] = np.array([1, 0])
    reprs[1] = np.array([0, 1])
if n_classes == 3:
    reprs[0] = np.array([1, 0])
    reprs[1] = np.array([1 / 2, np.sqrt(3) / 2])
    reprs[2] = np.array([1 / 2, -np.sqrt(3) / 2])
if n_classes == 4:
    reprs[0] = np.array([1, 0])
    reprs[1] = np.array([1 / np.sqrt(3), np.sqrt(2 / 3)])
    reprs[2] = np.array([1 / np.sqrt(3), np.exp(1j * 2 * np.pi / 3) * np.sqrt(2 / 3)])
    reprs[3] = np.array([1 / np.sqrt(3), np.exp(-1j * 2 * np.pi / 3) * np.sqrt(2 / 3)])

```

```

x, y = data
theta_aux = code_coords(theta, alpha, x)
C = circuit(theta_aux, entanglement)
psi = Statevector(C)
ans = np.sqrt(state_fidelity(reprs[y], psi))
return ans

```

Fidelity

$$-\frac{\sum_{i=1}^{N_{train}} \sqrt{\langle \psi_{true} | \psi_{meas,i} \rangle^2}}{N_{train}} = -\frac{\sum_{i=1}^{N_{train}} |\langle \psi_{true} | \psi_{meas,i} \rangle|}{N_{train}}$$

Maximally orthogonal true states.

```

x, y = d
theta_aux = code_coords(theta, alpha, x)
fids = mat_fidelities(theta_aux, weight, reprs, entanglement)
w_fid = w_fidelities(fids, weight)
if len(w_fid) == 4:
    Y = 1 / 3 * np.ones(len(w_fid))
    Y[y] = 1
if len(w_fid) == 3:
    Y = 1 / 4 * np.ones(len(w_fid))
    Y[y] = 1
if len(w_fid) == 2:
    Y = np.zeros(len(w_fid))
    Y[y] = 1
return 0.5 * np.linalg.norm(w_fid - Y) ** 2

```

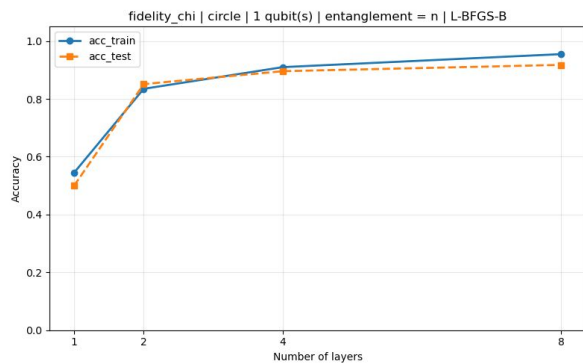
Weighted fidelity

$$\chi_{wf}^2(\vec{\alpha}, \vec{\theta}, \vec{w}) = \frac{1}{2} \sum_{\mu=1}^M \left(\sum_{c=1}^c \left(\alpha_c F_c(\vec{\theta}, \vec{w}, \vec{x}_{\mu}) - Y_c(\vec{x}_{\mu}) \right)^2 \right)$$

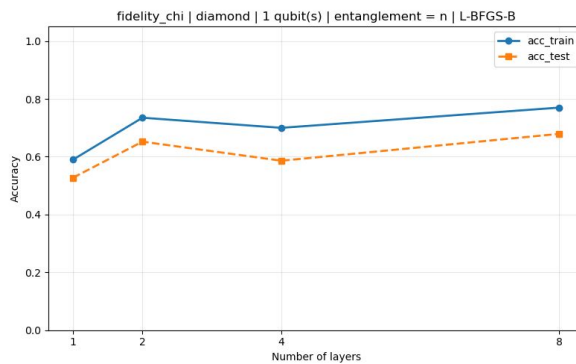
Single Qubit Results: not weighted fidelity

params = #qubits x #layers x 3 x 2

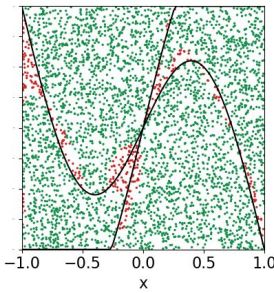
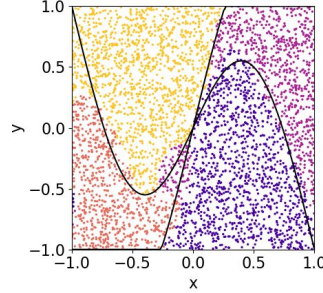
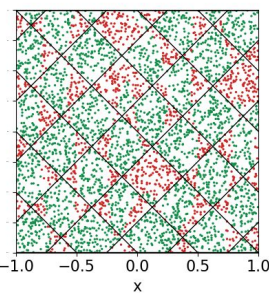
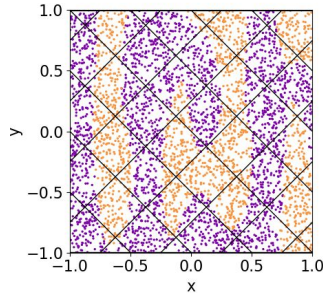
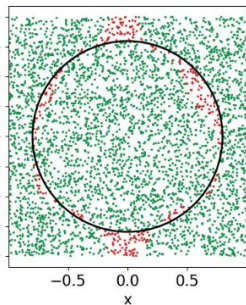
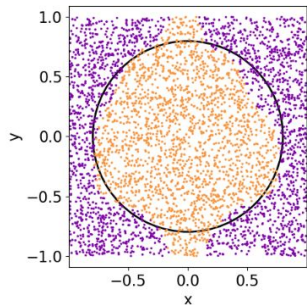
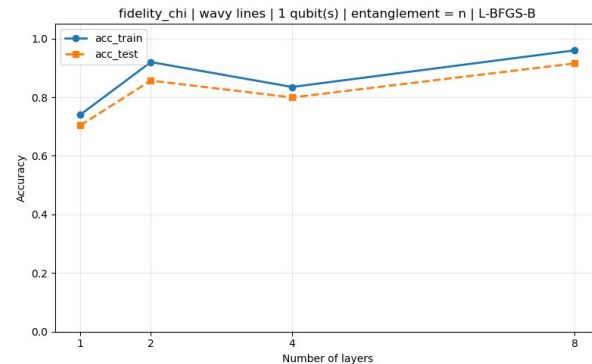
Circle



Diamond



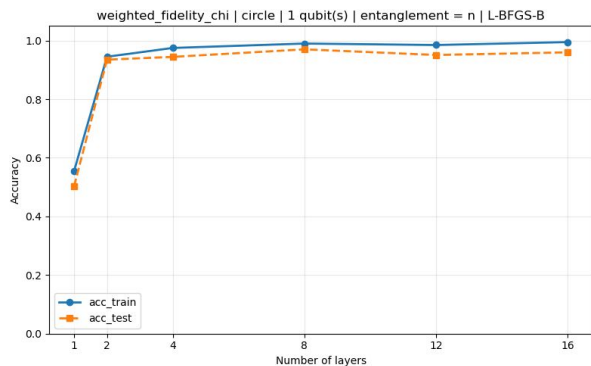
Wavy lines



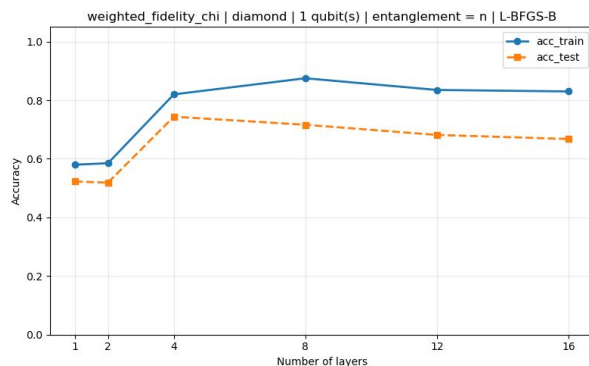
Single Qubit Results: weighted fidelity

params = #qubits x #layers x 3 x 2 x #classes

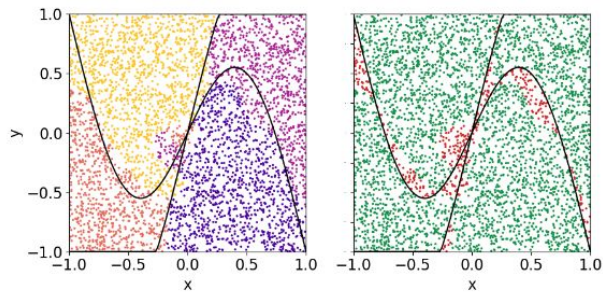
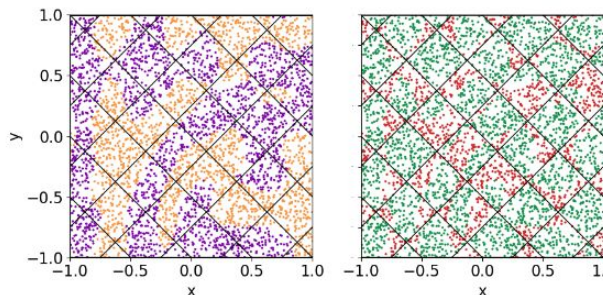
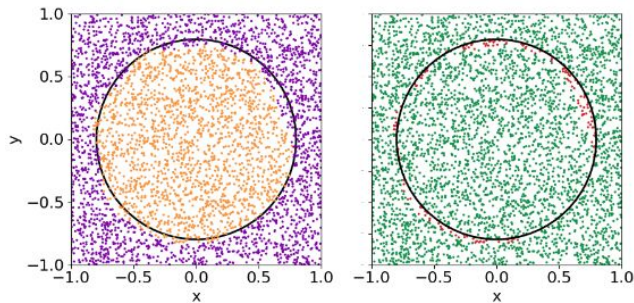
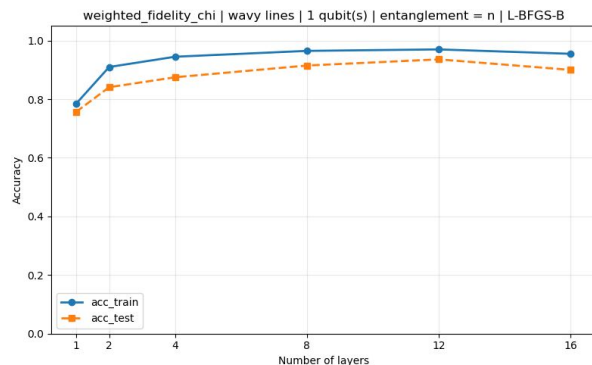
Circle



Diamond

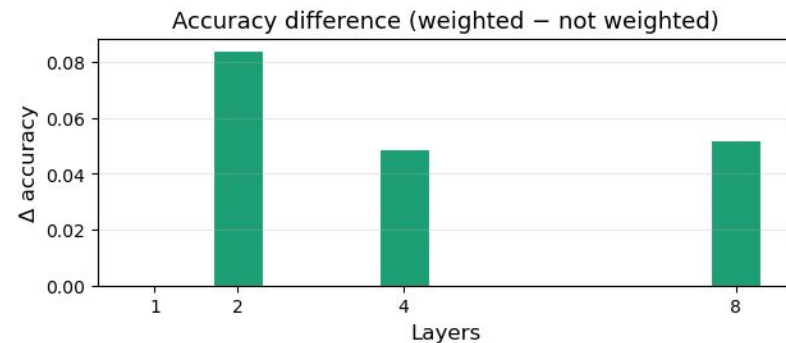
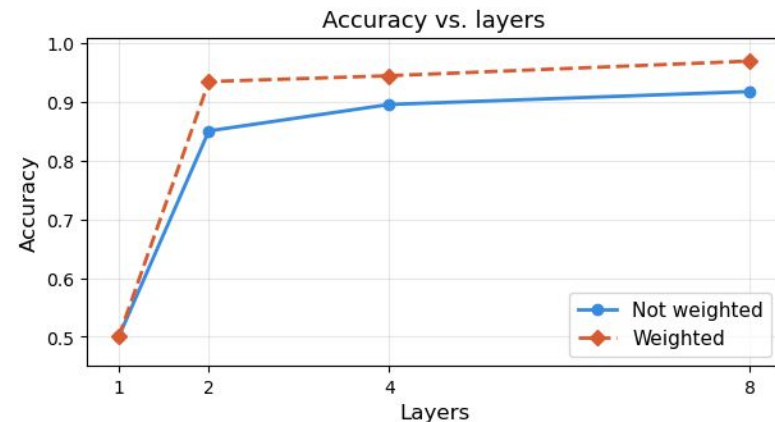


Wavy lines



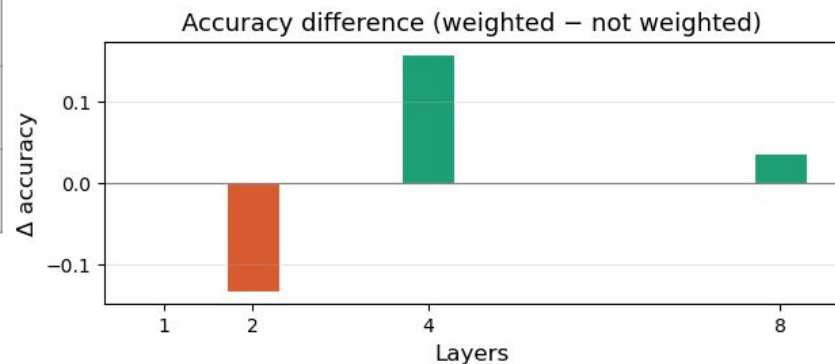
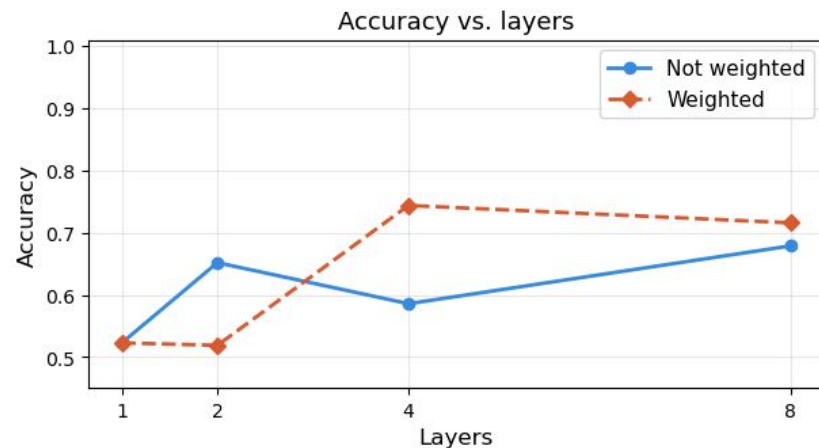
Circle - 1 qubit

# layers	not weighted		weighted	
	# params	accuracy test	# params	accuracy
1	6	0.5	12	0.5
2	12	0.851	24	0.935
4	24	0.896	48	0.945
8	48	0.918	96	0.970 !
12	-	-	144	0.951
16	-	-	192	0.96



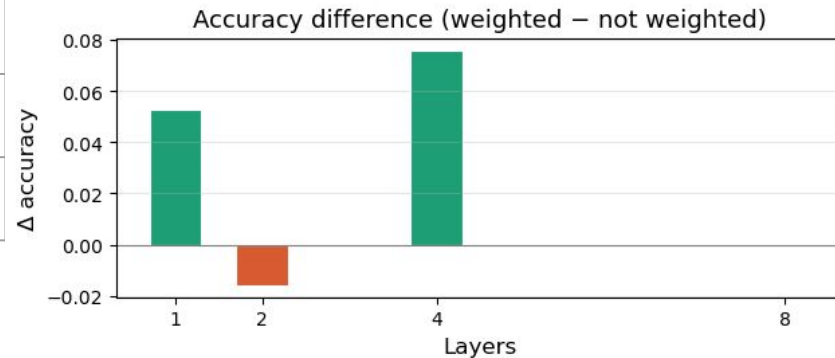
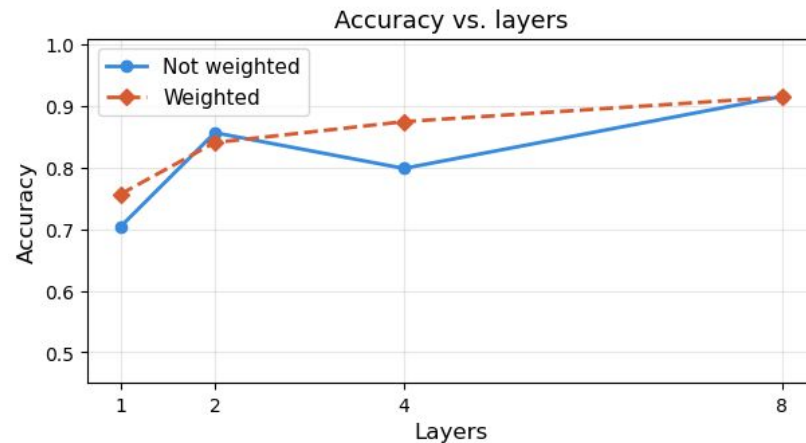
Diamond - 1 qubit

# layers	not weighted		weighted	
	# params	accuracy	# params	accuracy
1	6	0.523	12	0.523
2	12	0.652	24	0.519
4	24	0.586	48	0.744 !
8	48	0.679	96	0.716
12	-	-	144	0.682
16	-	-	192	0.668



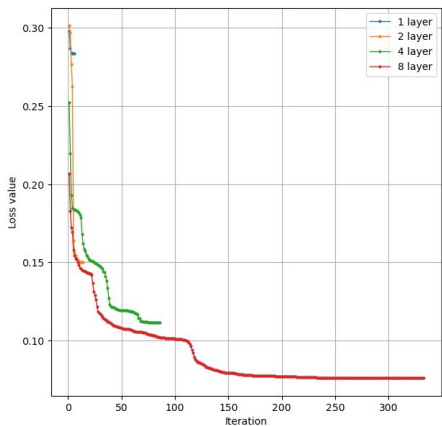
Wavy lines - 1 qubit

# layers	not weighted		weighted	
	# params	accuracy	# params	accuracy
1	6	0.704	18	0.757
2	12	0.857	36	0.841
4	24	0.799	72	0.875
8	48	0.916	166	0.915
12	-	-	216	0.936
16	-	-	288	0.9

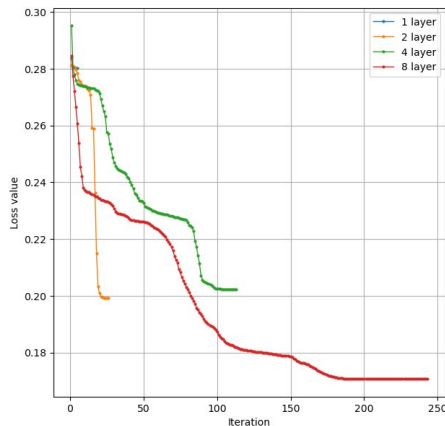


Loss drop

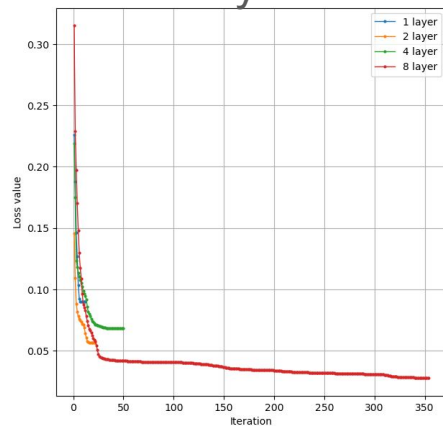
Circle



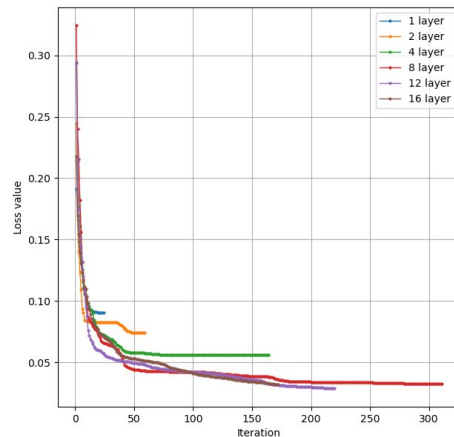
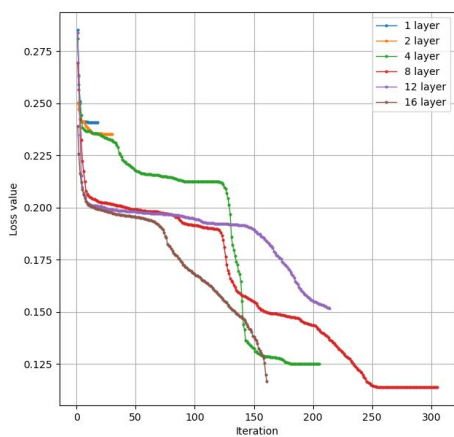
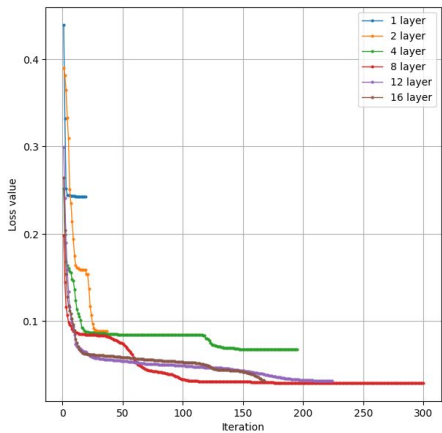
Diamond



Wavy lines



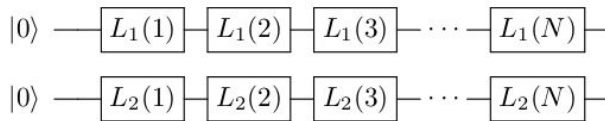
Not weighted



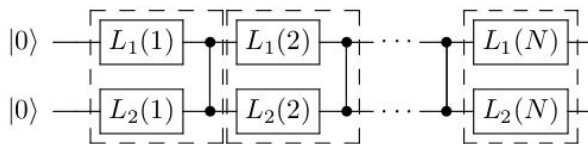
Weighted

Multi qubits - entanglement implementation

2 qubits



(a) Ansatz with no entanglement



(b) Ansatz with entanglement

Just using weighted fidelity,
focusing on one qubit by time:

$$F_{c,q}(\vec{\theta}, \vec{w}, \vec{x}) = \langle \tilde{\psi}_c | \rho_q(\vec{\theta}, \vec{w}, \vec{x}) | \tilde{\psi}_c \rangle,$$

$$\chi_{wf}^2(\vec{\alpha}, \vec{\theta}, \vec{w}) = \frac{1}{2} \sum_{\mu=1}^M \sum_{c=1}^C \left(\sum_{q=1}^Q \left(\alpha_{c,q} F_{c,q}(\vec{\theta}, \vec{w}, \vec{x}_\mu) - Y_c(\vec{x}_\mu) \right)^2 \right)$$

```
def _qcircuit_2qubit_noentanglement(theta_aux):
    num_qubits = theta_aux.shape[0]
    num_layers = theta_aux.shape[1]

    qc = QuantumCircuit(num_qubits)

    for l in range(num_layers):
        for q in range(num_qubits):

            mat = U3(theta_aux[q, l, :])

            gate = UnitaryGate(mat, label="U3_custom")
            qc.append(gate, [q])

    return qc
```

```
def _qcircuit_2qubit_entanglement(theta_aux):
    qc = QuantumCircuit(2)
    for l in range(theta_aux.shape[1] - 1):
        for q in range(theta_aux.shape[0]):
            mat = U3(theta_aux[q, l, :])
            qc.append(UnitaryGate(mat), [q])

            qc.cz(0,1)
        for q in range(theta_aux.shape[0]):
            mat = U3(theta_aux[q, -1, :])
            qc.append(UnitaryGate(mat), [q])
    return qc
```

3 qubits

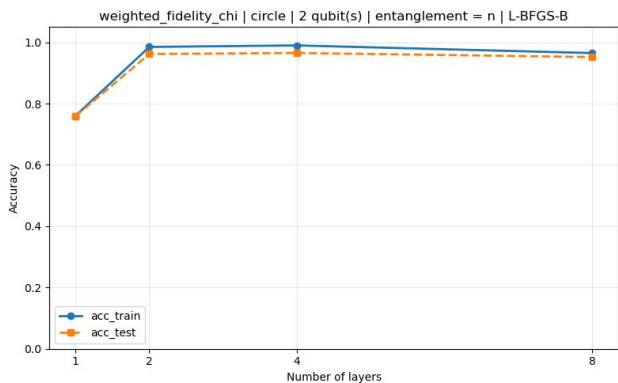
```
def _qcircuit_3qubit_noentanglement(theta_aux):  
    qc = QuantumCircuit(3)  
    for l in range(theta_aux.shape[1]):  
        for q in range(3):  
            mat = U3(theta_aux[q, l, :])  
            gate = UnitaryGate(mat, label="U3_custom")  
            qc.append(gate, [q])  
    return qc
```

```
def _qcircuit_3qubit_entanglement(theta_aux):  
    qc = QuantumCircuit(3)  
    for l in range(theta_aux.shape[1] - 1):  
        for q in range(theta_aux.shape[0]):  
            mat = U3(theta_aux[q, l, :])  
            qc.append(UnitaryGate(mat), [q])  
        if l%2 == 0:  
            qc.cz(0,1) ←  
            qc.cz(1,2) ←  
        elif l%2 == 1:  
            qc.cz(0,2) ←  
    for q in range(theta_aux.shape[0]):  
        mat=U3(theta_aux[q, -1, :])  
        qc.append(UnitaryGate(mat), [q])  
    return qc
```

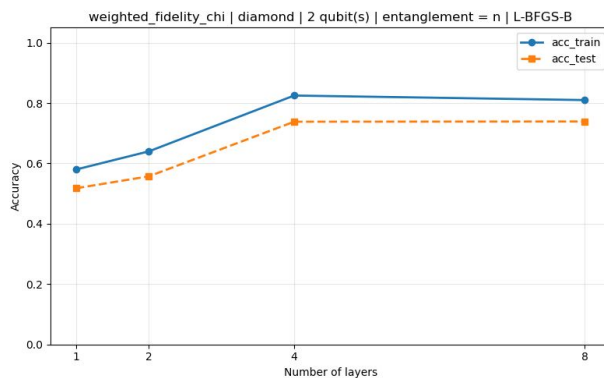
2 Qubits Results: not entangled, weighted fidelity

params = #qubits x #layers x 3 x 2 x #classes

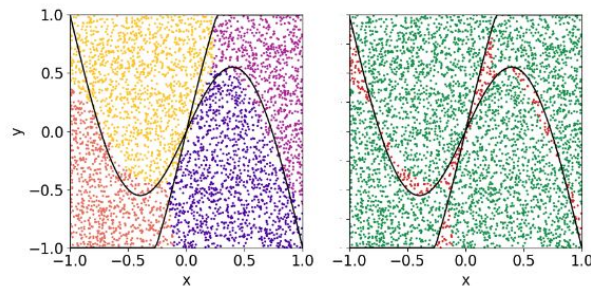
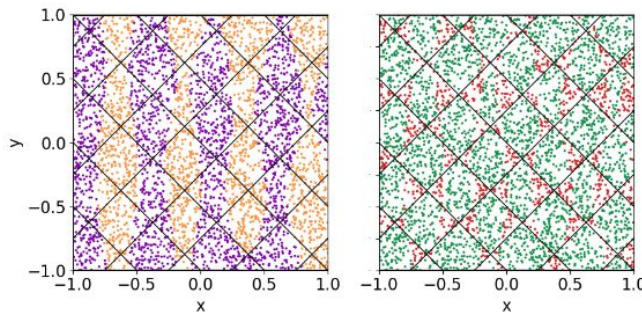
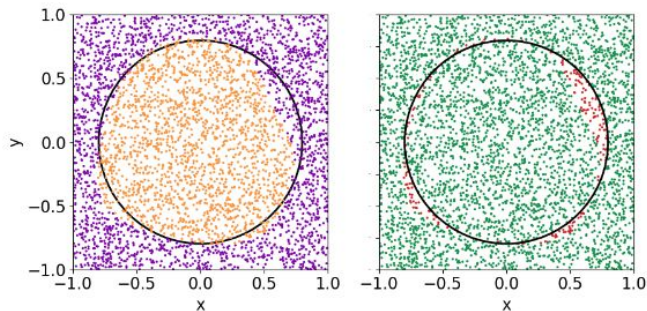
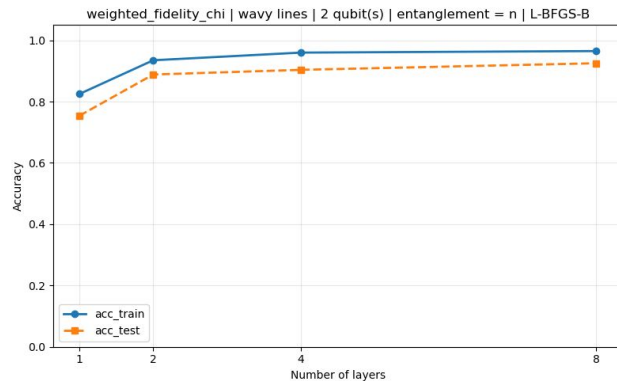
Circle



Diamond



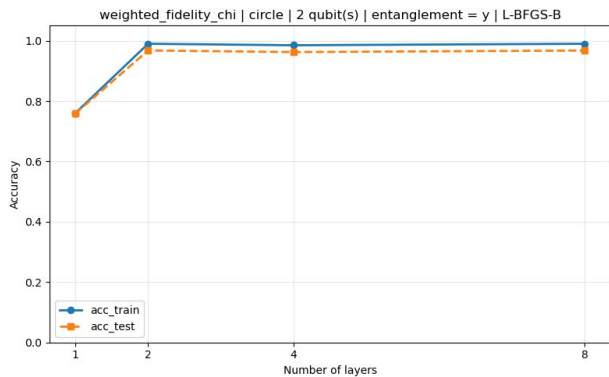
Wavy lines



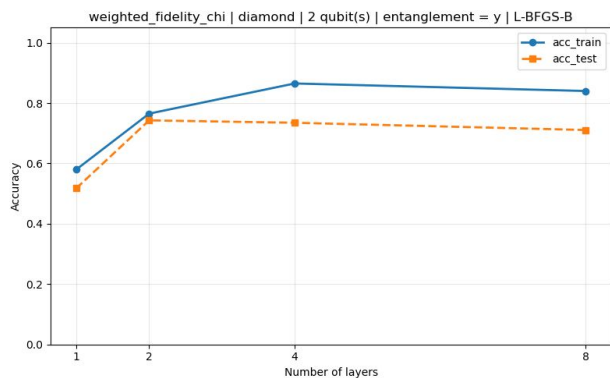
2 Qubits Results: entangled, weighted fidelity

params = #qubits x #layers x 3 x 2 x #classes

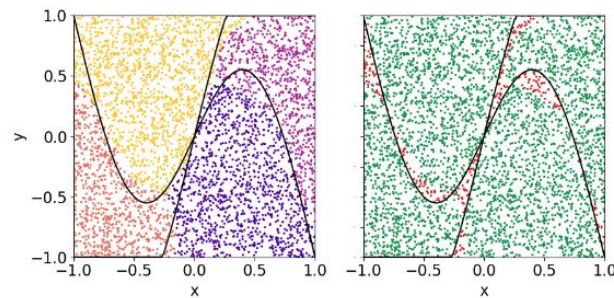
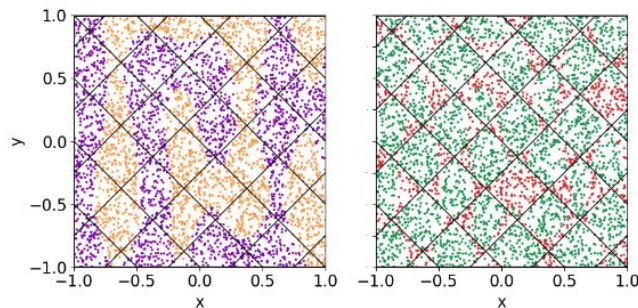
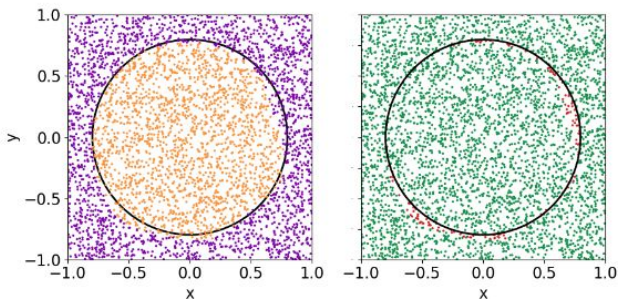
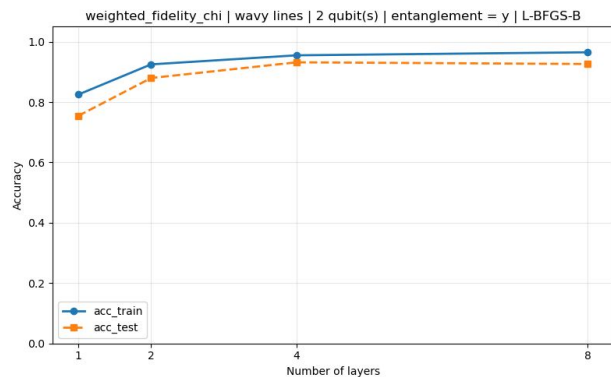
Circle



Diamond

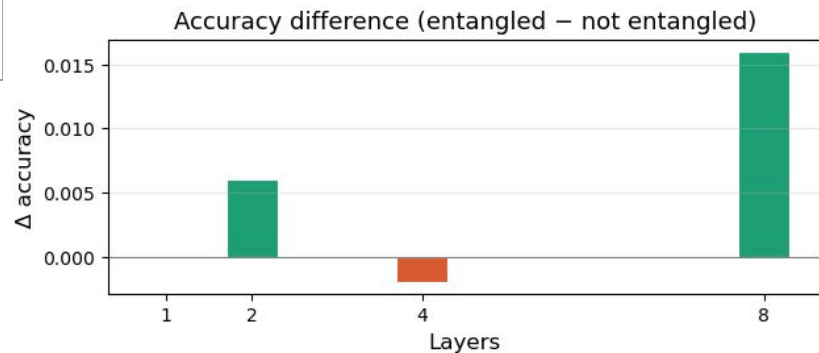
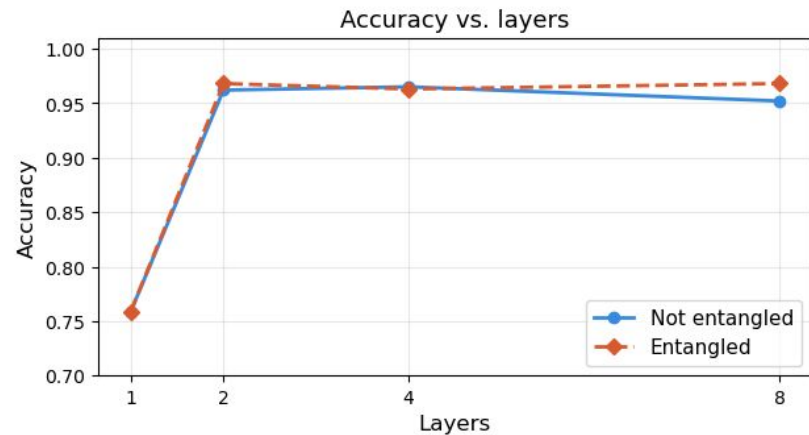


Wavy lines



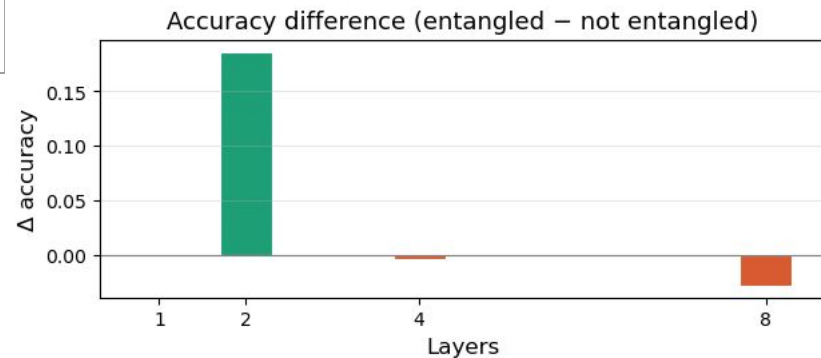
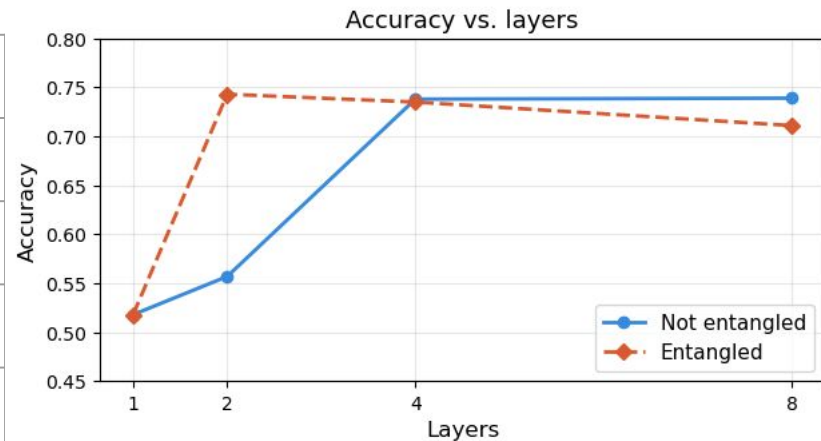
Circle - 2 qubits

# layers	not entangled		entangled	
	# params	accuracy test	# params	accuracy
1	24	0.759	24	0.759
2	48	0.962	48	0.968
4	96	0.965	96	0.963
8	192	0.952	192	0.968



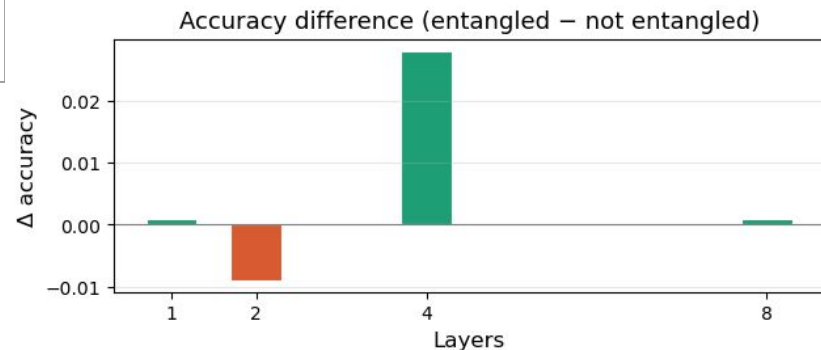
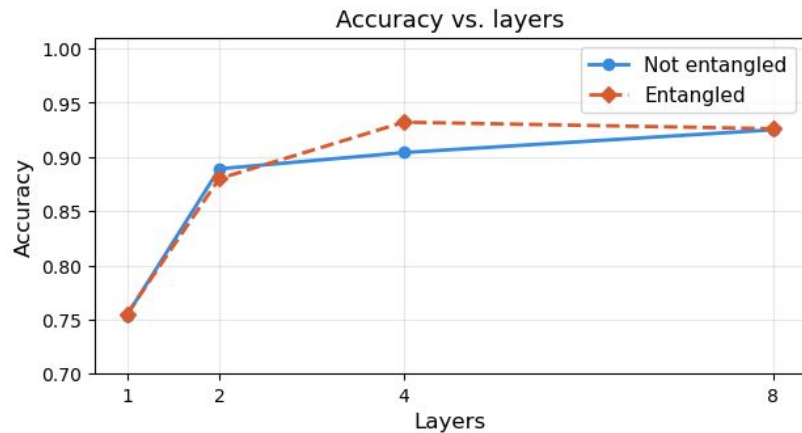
Diamond - 2 qubits

# layers	not entangled		entangled	
	# params	accuracy test	# params	accuracy
1	24	0.518	24	0.518
2	48	0.557	48	0.743
4	96	0.738	96	0.735
8	192	0.739	192	0.711



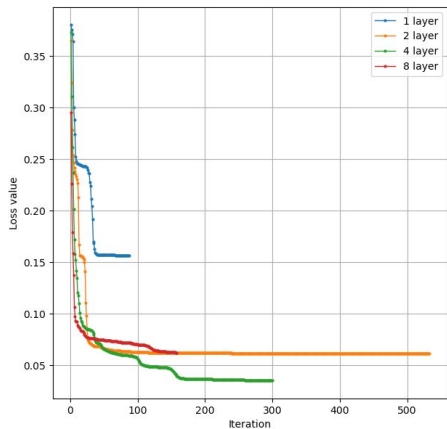
Wavy lines - 2 qubits

# layers	not entangled		entangled	
	# params	accuracy test	# params	accuracy
1	48	0.754	48	0.755
2	96	0.889	96	0.880
4	192	0.904	192	0.932
8	384	0.925	384	0.926

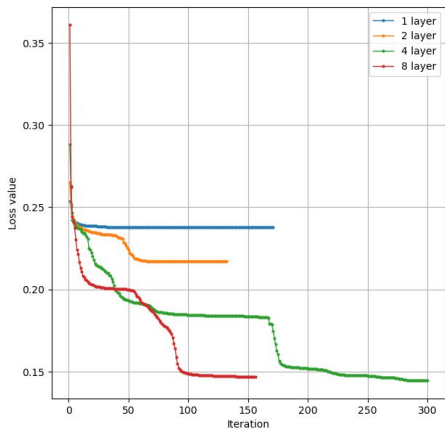


Loss drop

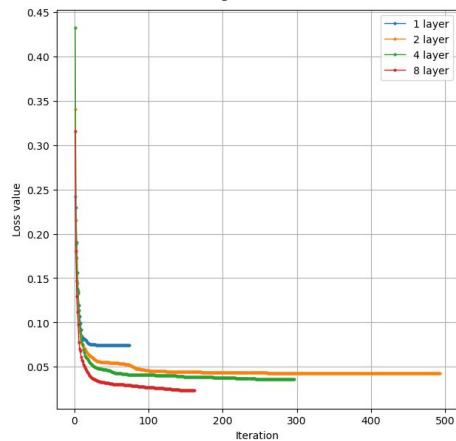
Circle



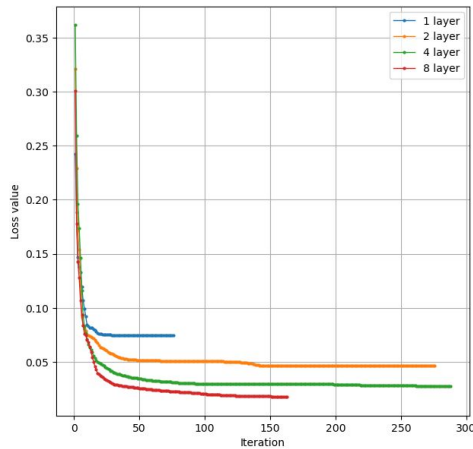
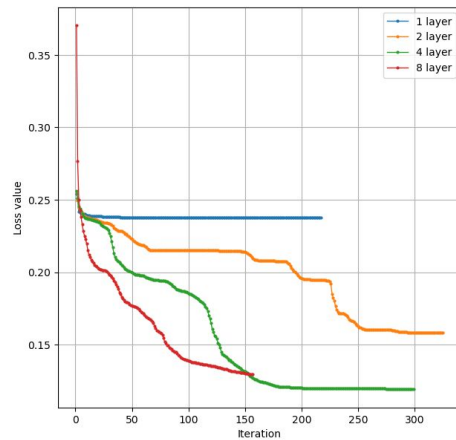
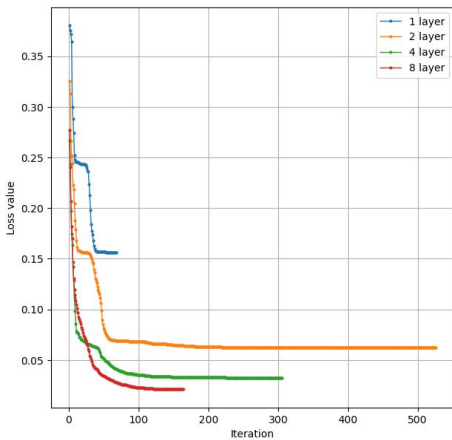
Diamond



Wavy lines



Not entangled

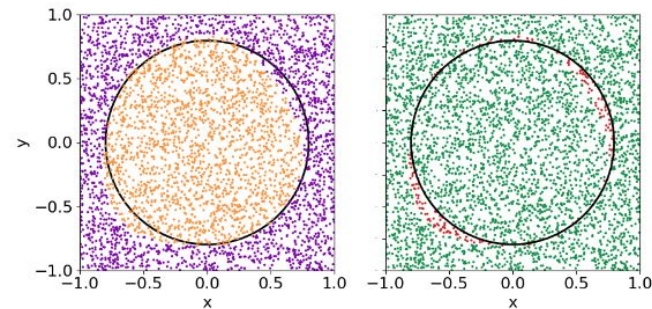
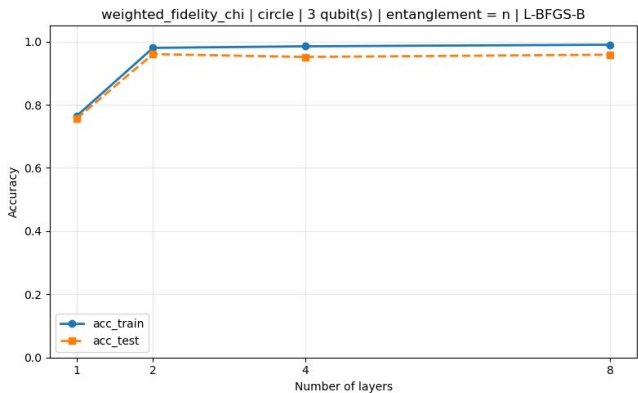


Entangled

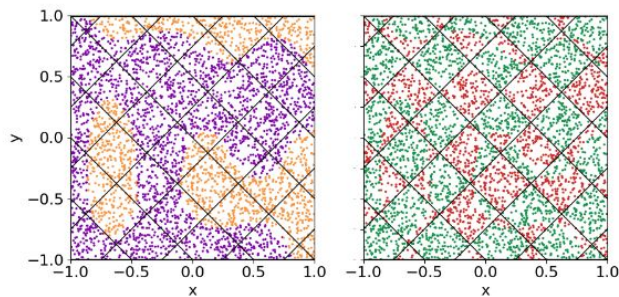
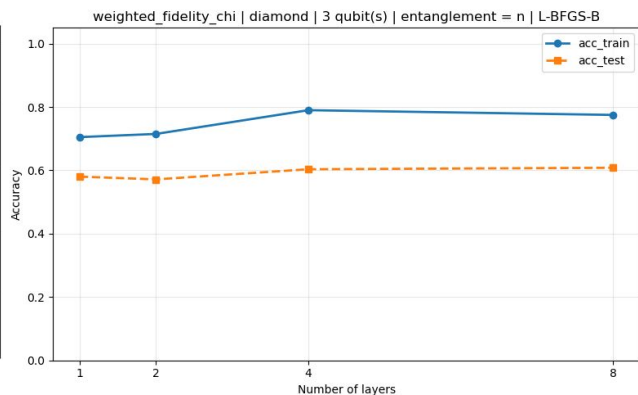
3 Qubits Results: not entangled, weighted fidelity

params = #qubits x #layers x 3 x 2 x #classes

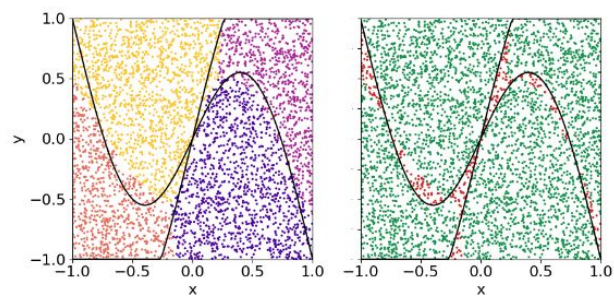
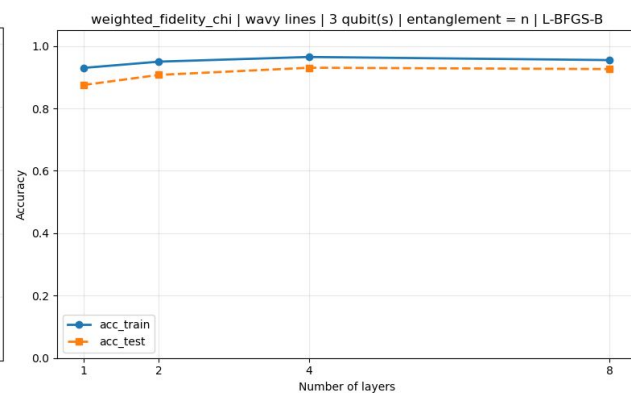
Circle



Diamond



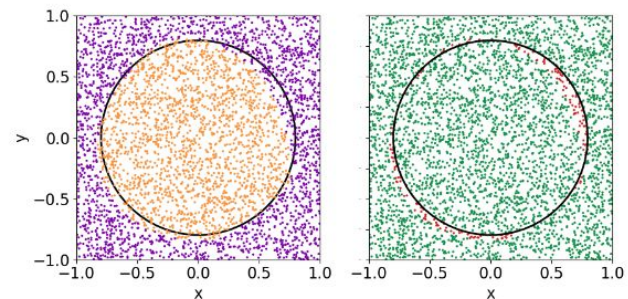
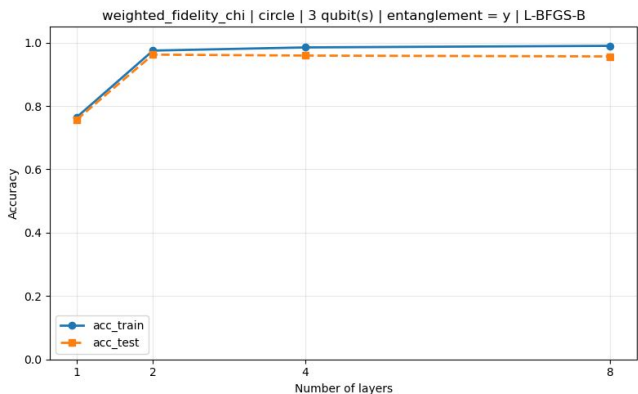
Wavy lines



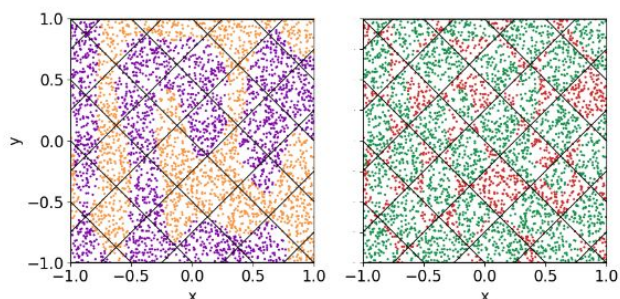
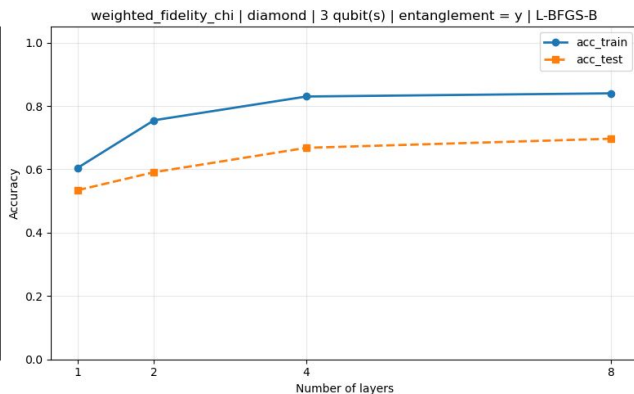
3 Qubits Results: entangled, weighted fidelity

params = #qubits x #layers x 3 x 2 x #classes

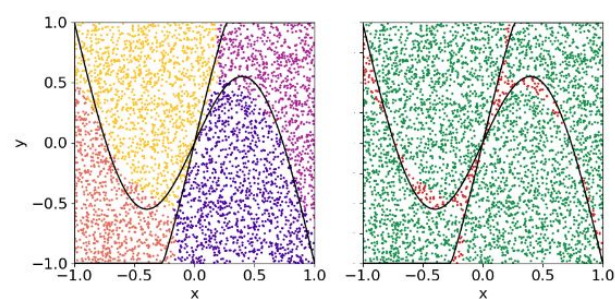
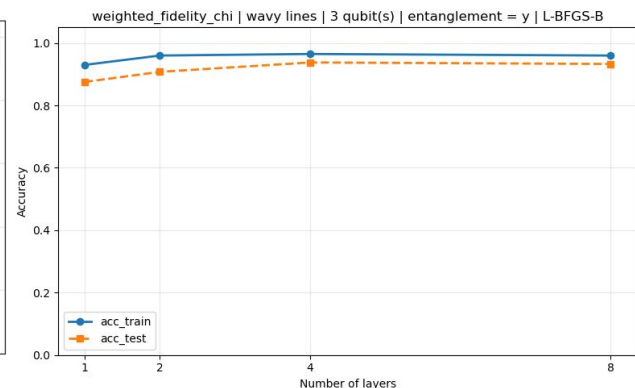
Circle



Diamond

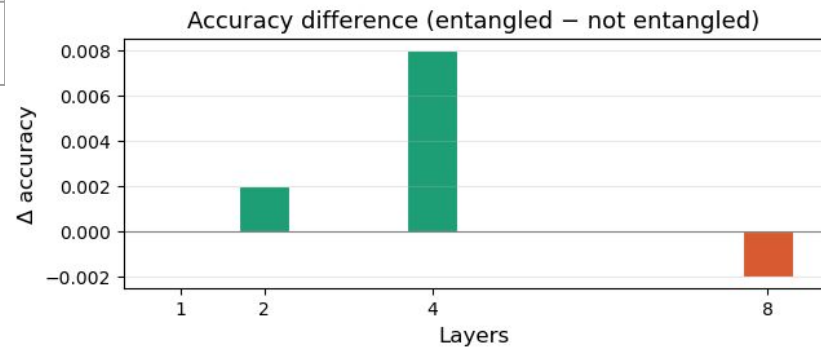
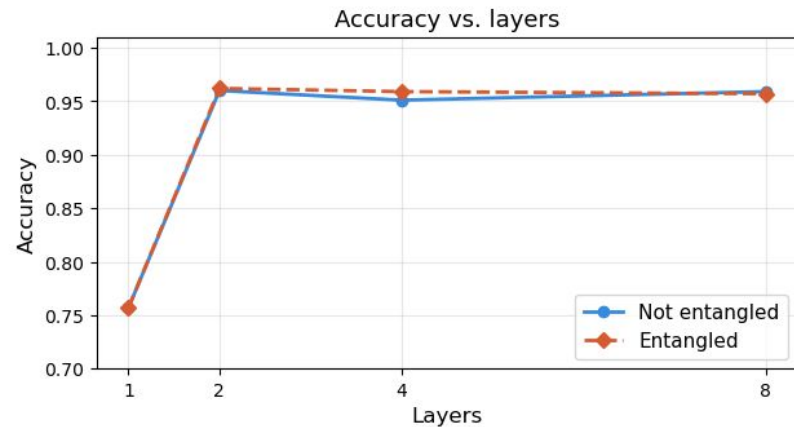


Wavy lines



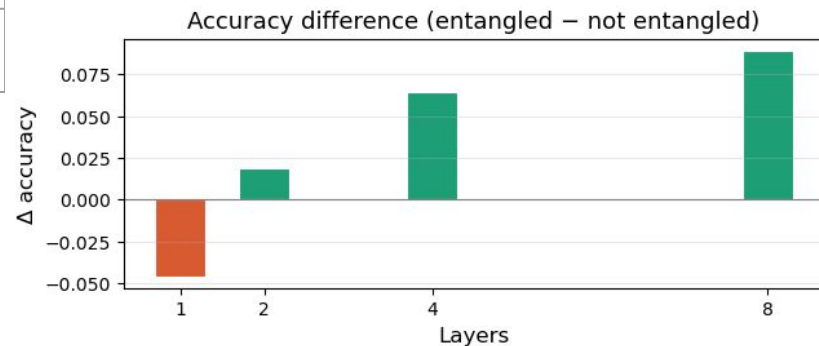
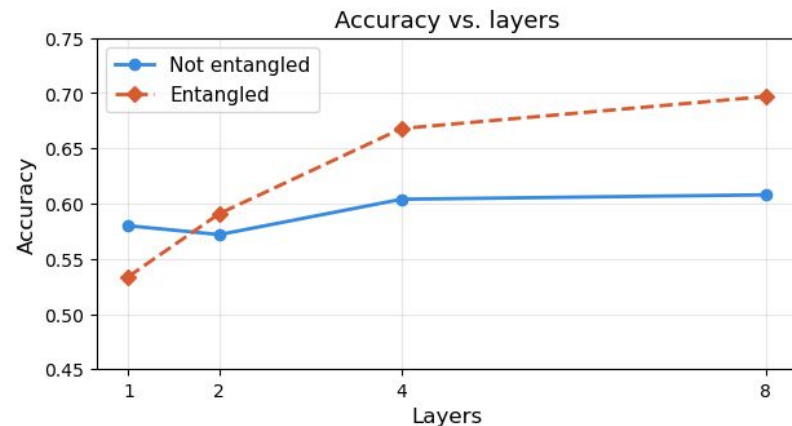
Circle - 3 qubits

# layers	not entangled		entangled	
	# params	accuracy test	# params	accuracy
1	36	0.757	36	0.757
2	72	0.960	72	0.962
4	144	0.951	144	0.959
8	288	0.959	288	0.957



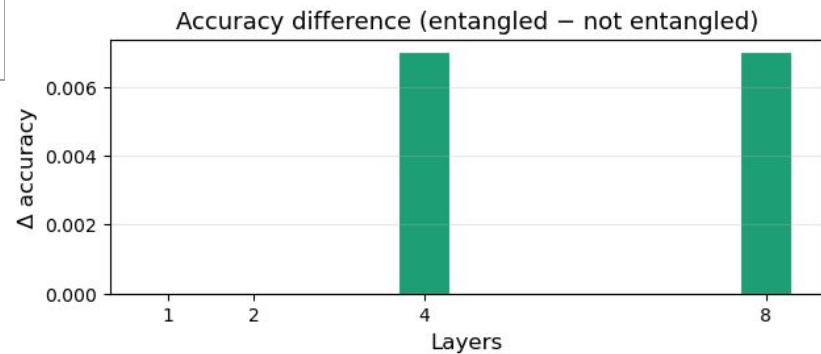
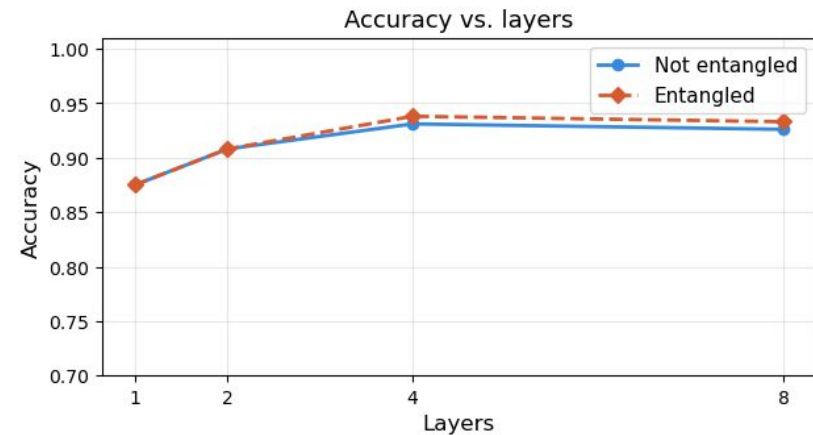
Diamond - 3 qubits

# layers	not entangled		entangled	
	# params	accuracy test	# params	accuracy
1	36	0.580	36	0.534
2	72	0.572	72	0.591
4	144	0.604	144	0.668
8	288	0.608	288	0.697



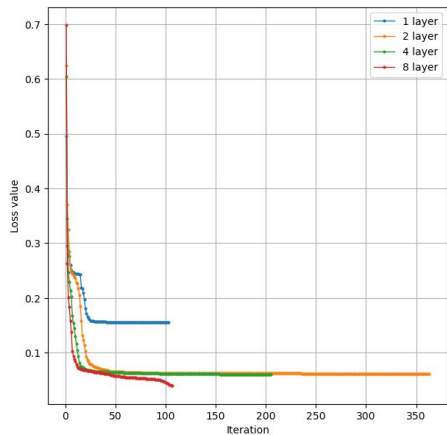
Wavy lines - 3 qubits

# layers	not entangled		entangled	
	# params	accuracy test	# params	accuracy
1	72	0.875	72	0.875
2	144	0.908	144	0.908
4	288	0.931	288	0.938 !
8	576	0.926	576	0.933

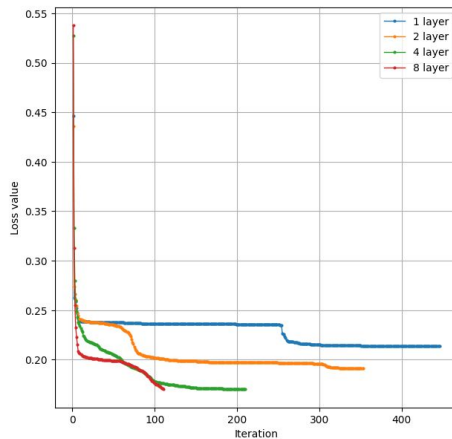


Loss drop

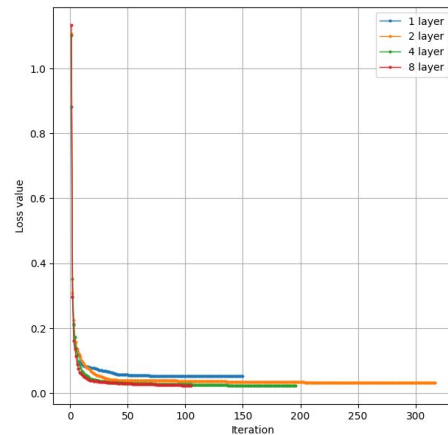
Circle



Diamond

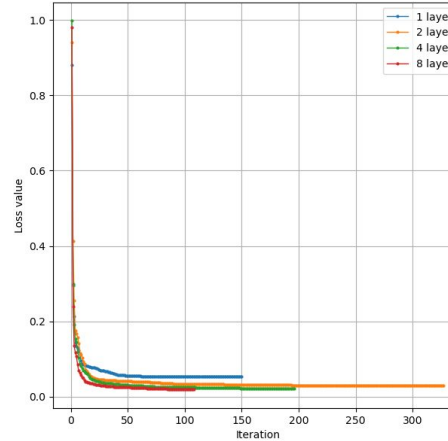
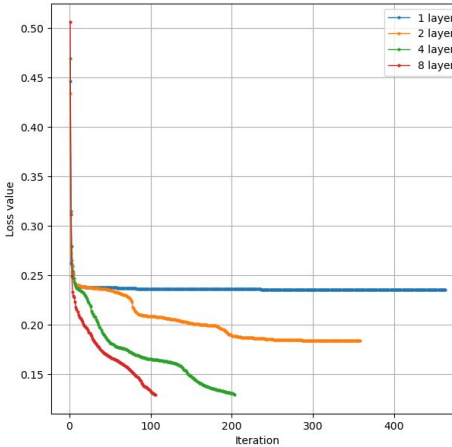
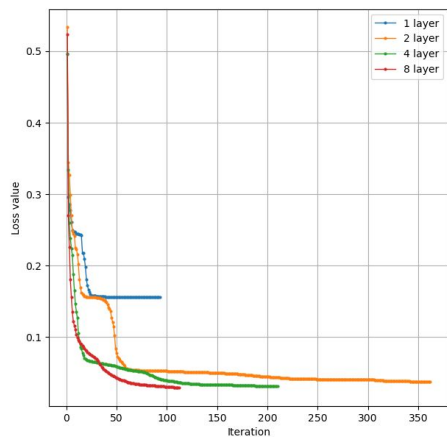


Wavy lines



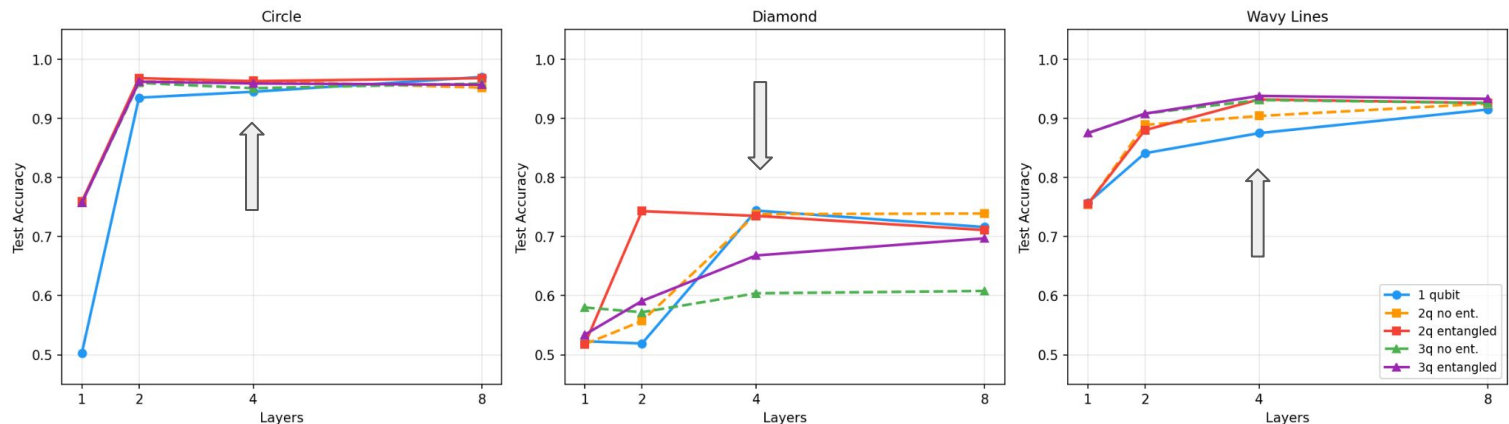
Not entangled

Entangled



Accuracy evolution for the datasets

Test Accuracy vs. Number of Re-uploading Layers — All Datasets



Results
considering
4 layers

Circuit	Accuracy circle	Accuracy diamond	Accuracy Wavy lines
1 qubit	0.9445	0.7438	0.8750
2 qubits n.ent	0.9625	0.7348	0.9318
2 qubits ent	0.96525	0.7383	0.9038
3 qubits n.ent	0.95125	0.6035	0.9305
3 qubits ent	0.95925	0.6680	0.9378

Multi-qubit circuits and depth required

Circle

- ~93.5% accuracy achieved with **1 qubit and only 2 layers**.
- **Multi-qubit circuits** reach even higher performance (~96%) with **2 layers**.
- Increasing circuit depth provides **little additional improvement** (stable)

Wavy Lines

- Accuracy increases significantly with additional qubits:
 - **1 qubit, 1 layer:** 75.7%
 - **3 qubits, 1 layer:** 87.5%
- Multi-qubit circuits converge faster to high accuracy:
 - **>90% accuracy** achieved with:
 - 2 layers (3 qubits)
 - 4 layers (2 qubits)
 - 8 layers (1 qubit)

Multi-qubit circuits can reduce the required depth for datasets with complex decision boundaries (wavy lines), but this is not a universal advantage.

Diamond

- Most challenging dataset due to its **high-frequency decision boundary changes**.
- **~74% accuracy** achieved with:
 - 1 qubit after 4 layers
 - 2 entangled qubits after only 2 layers
- However, these gains are **not consistently sustained**.
- Performance plateaus around **70–74%**, and additional depth or qubits do not substantially improve results

Entanglement and Layers accuracy impact

- Differences between entangled (Y) and non-entangled (N) circuits are generally **below 2%**.
- In some cases, **non-entangled circuits outperform entangled ones** (e.g., *Wavy Lines* with 2 qubits).
- For the *Diamond* dataset with 3 qubits, entanglement provides a more noticeable benefit:
 - **60.3% → 66.8% at 4 layers.**

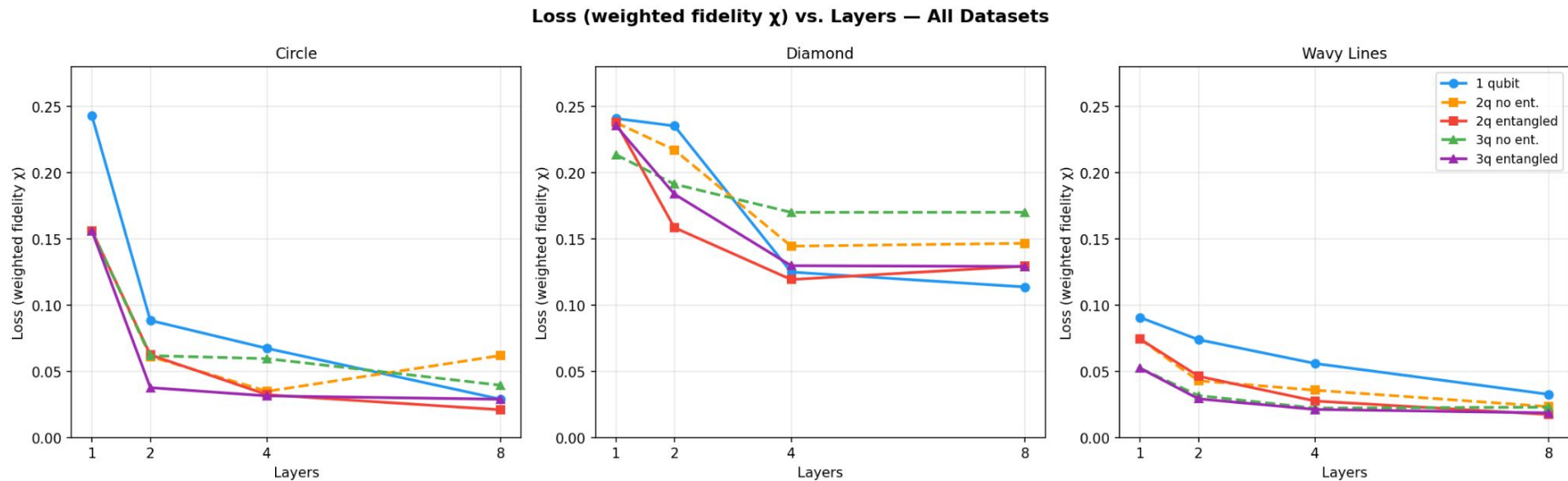
Entanglement does not guarantee consistent performance improvements. Its benefits appear only in specific cases, particularly for more challenging datasets with a larger number of qubits. The data re-uploading architecture alone is already capable of capturing most relevant correlations.

Pattern observed across all three datasets:

- **Layer 1 → 2:** Largest accuracy improvement across all circuits and datasets.
 - Example: *Circle* (1 qubit): **50.3% → 93.5%** test accuracy.
- **Layer 2 → 4:** Smaller but still consistent improvements.
 - Example: *Wavy Lines* (1 qubit): **84.1% → 87.5%**.
- **Layer 4 → 8:** Little to no improvement; in some cases, a slight decrease is observed (possible mild overfitting).
- **Layer 8 → 12 → 16** (1-qubit circuits only): Accuracy remains stable with minor fluctuations.

Rapid gains in the first layers, followed by saturation.

Minimal loss achieved

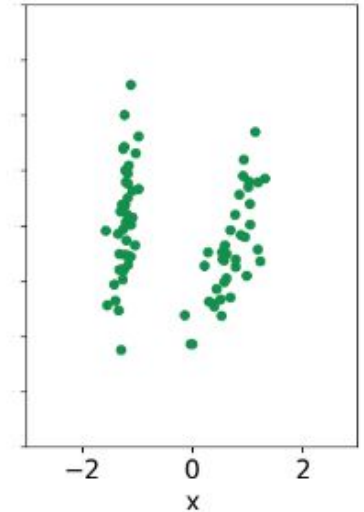
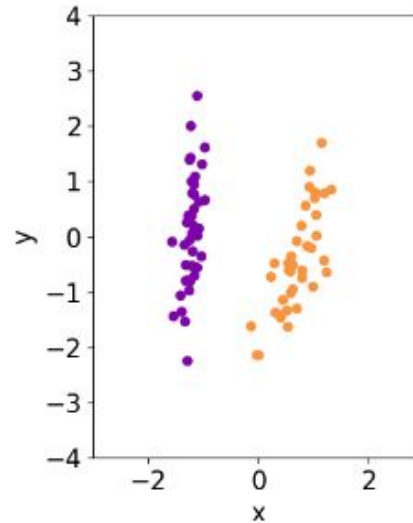
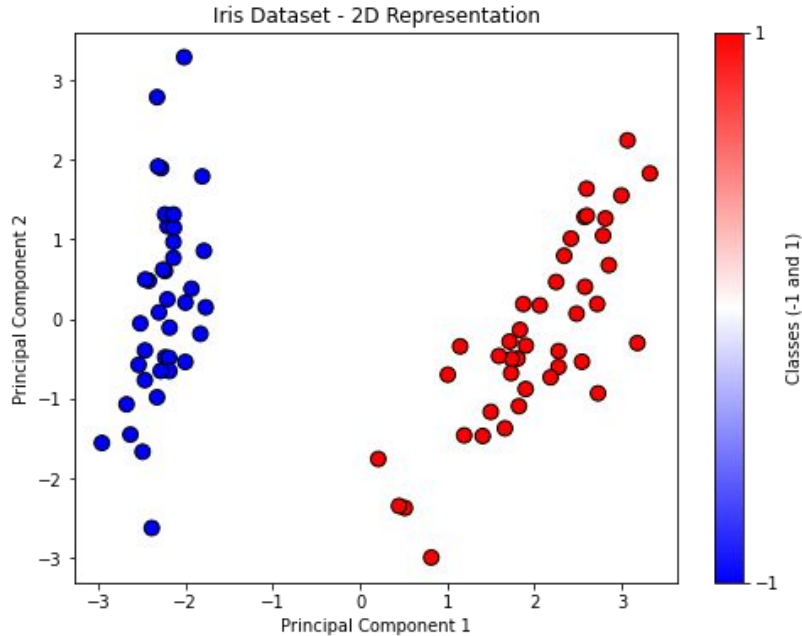


Loss drops sharply from layer 1 to 2 across all datasets and circuits, then flattens out. Multi-qubit circuits converge faster and reach lower loss values than single-qubit at low layer counts, especially on Diamond. Diamond shows the highest residual loss overall, consistent with it being the hardest decision boundary to learn. From layer 4 onward, all circuits plateau with minimal further gains. Entangled and non-entangled variants perform similarly, with only minor divergence on Diamond (3q).

IRIS quantum results:

Setosa vs versicolor: easy problem

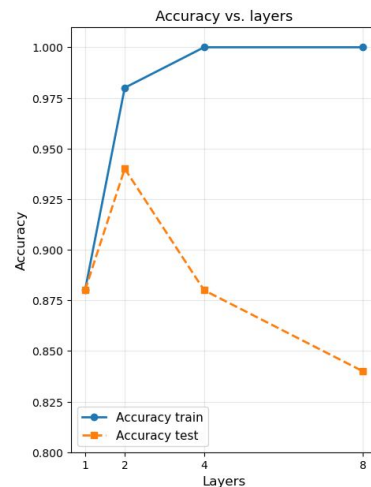
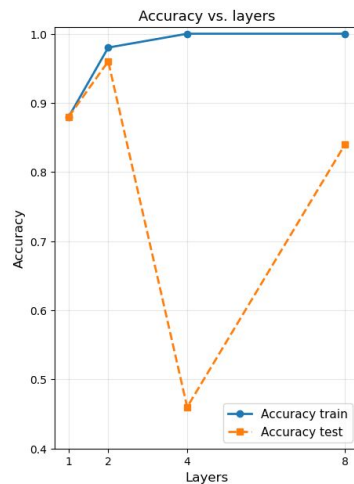
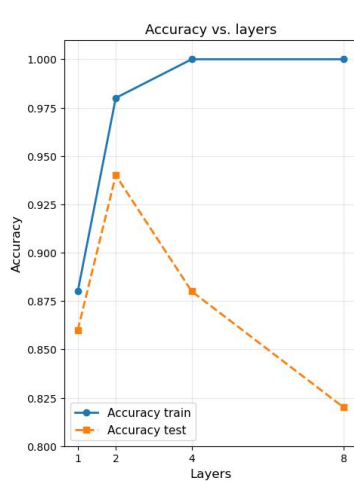
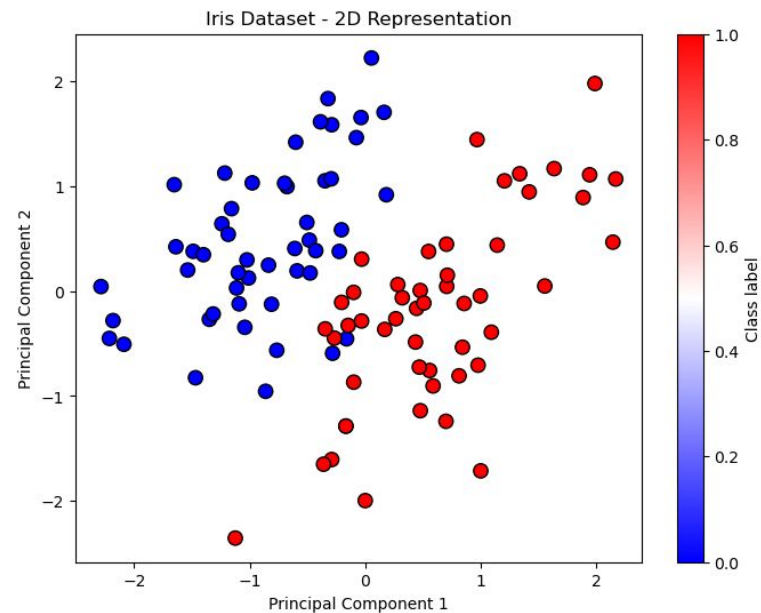
Accuracy test = 1.0 starting from 1 qubit 1 layer. Probably due to the good separation between the 2 features.



IRIS quantum results:

versicolor vs virginica: more complex problem

probably overfitting!

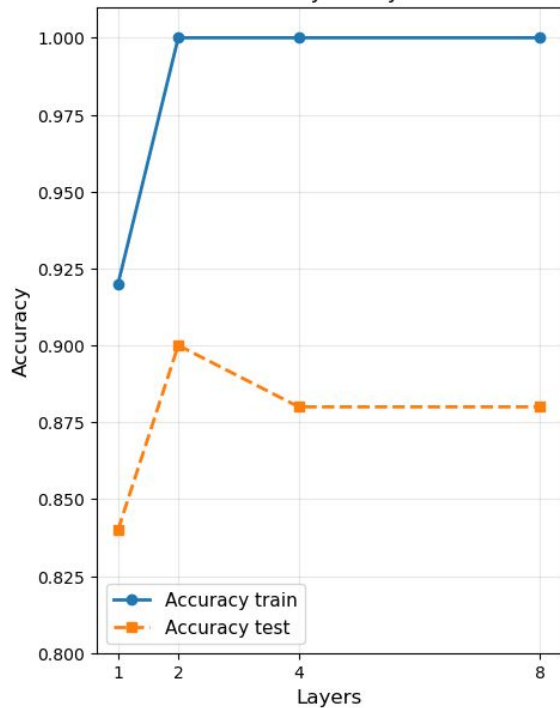


Retry with 50 train - 50 test...

... but did not change too much

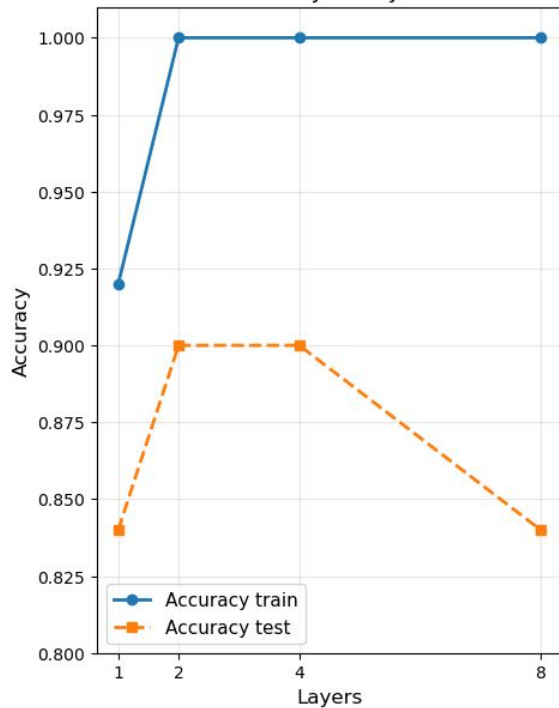
1 qubit

Accuracy vs. layers



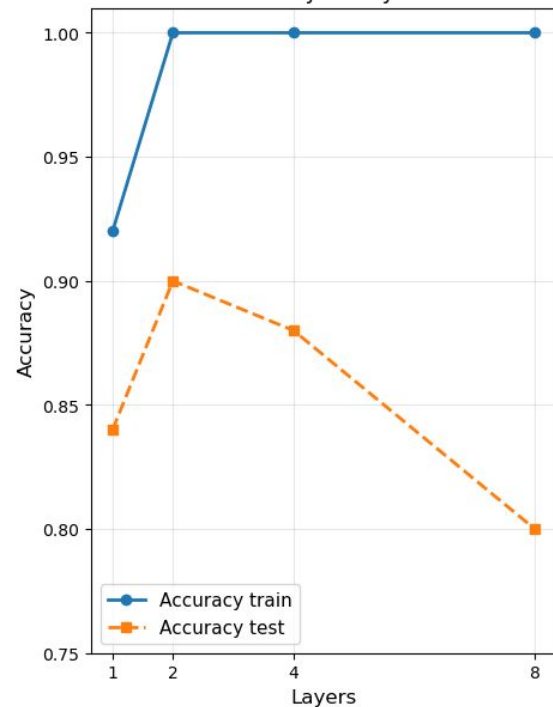
2 qubits no ent

Accuracy vs. layers

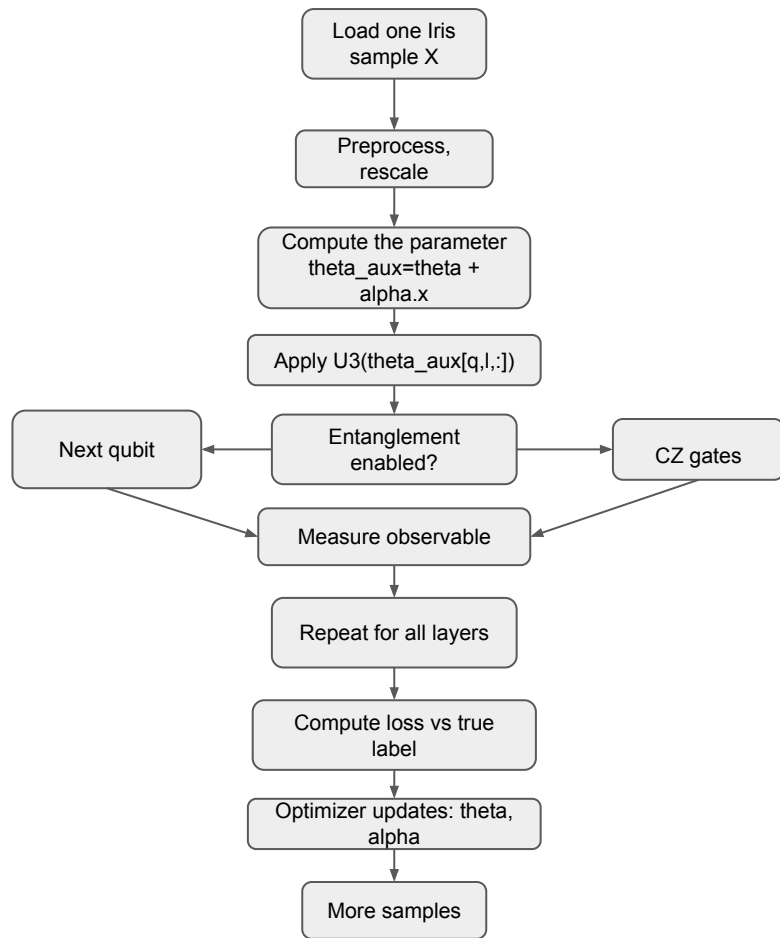


2 qubits ent

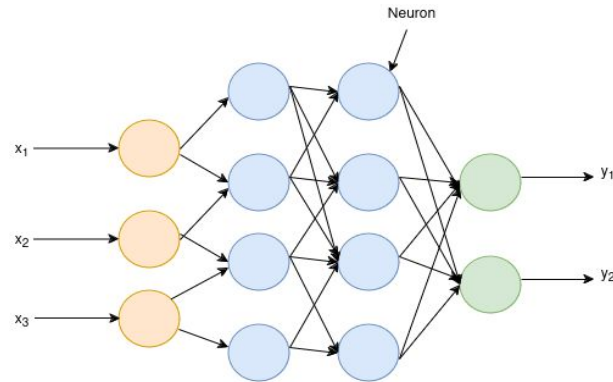
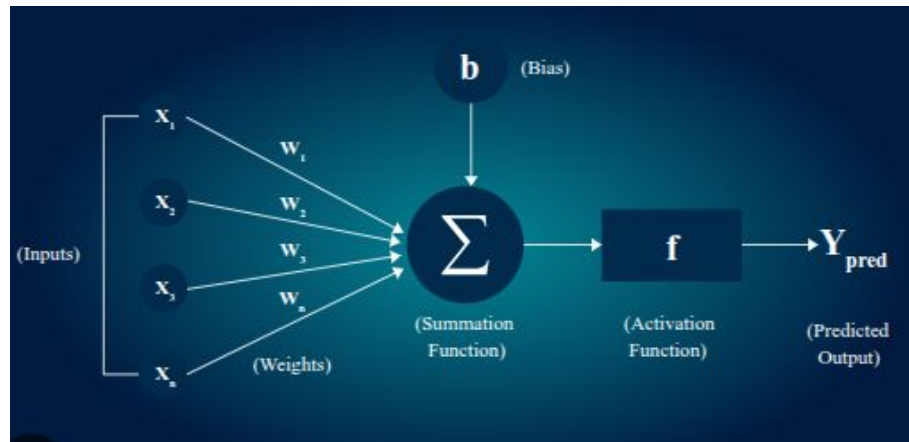
Accuracy vs. layers



Quantum Architecture



Neural Network Architecture



Architecture comparison

- Sequential vs parallel processing of data points
- Number of qubits—>circuit width; layers—>depth of the circuit; reuploading
In NN it is the number of hidden layers and no of neurons in each hidden layer.
- Neuron vs the Rotation encoding as input
- Trainable parameters: theta, alpha vs weights and biases
- Update rule: SGD for NN and Minimize method from Scipy

Computational Time Complexity Comparison

Theta Encoding	$O(QLd)$
U3 Unitary matrix	$O(1)$
Circuit definition	$O(QL)$
State Vector Simulation *	$O(2^Q)$
Minimization	$O(M(QL+2^Q))$

Layer (type)	Output Shape	Param #
dense_26 (Dense)	(None, 64)	192
dense_27 (Dense)	(None, 16)	1,040
dense_28 (Dense)	(None, 1)	17

$$T_{\text{forward}} = O(2 \cdot 64 + 64 \cdot 16 + 16 \cdot 1)$$

* [Efficient Classical Simulation of the Quantum Random Access Memory](#)

Iris in the re-uploading algorithm

For each Iris point $x \in \mathbb{R}^2$ the model builds

$$\theta_{\text{aux}}(x) = \theta + \alpha \circ x,$$

then applies the unitary circuit

$$|\psi(x)\rangle = U_{\square}(x) \cdots U_1(x)|0\rangle$$

with each $U_{\square}(x)$ constructed from U_3 gates and optional CZ entanglement, and finally computes either a fidelity-based loss or a weighted fidelity loss. Parameters (θ, α) are updated by a classical optimizer such as L-BFGS-B until the training cost converges.

Conclusion

A single-qubit data re-uploading circuit can learn nonlinear decision boundaries — but needs multiple layers to do so, since one layer only produces a linear separator; performance is strongest on datasets without high-frequency boundaries (Circle, Wavy Lines), and weaker on Diamond.

Multi-qubit circuits reduce the required depth mainly for complex boundaries (Wavy Lines), but this is not a universal advantage — for simpler datasets, 1 qubit catches up with enough layers.

Entanglement does not guarantee consistent improvement. Its benefit is dataset- and qubit-count-dependent, appearing mainly in harder cases (Diamond, 3 qubits) — the re-uploading structure alone already captures most relevant correlations.

Accuracy and loss both follow the same pattern with depth: rapid gains in the first 1–2 layers, followed by saturation — additional layers beyond 4 bring diminishing returns across all circuits and datasets.

The Quantum Classifier compared to Neural Network uses very few trainable parameters and has potential to represent complex nonlinear boundaries, but it is very slow compared to NN due to $O(2^Q)$ complexity caused by Qiskit State Vector Simulation. Despite it is capable of classifying real-world dataset such as Iris.

Circle

$x^2 + y^2 < r^2$ - quadratic hence
non-linear

Accuracy 8 Layers single qubit: 97%

Diamond

The decision regions are chequered
patterns, a single decision boundary
cannot separate it.

Accuracy 4 Layers single qubit: 74%

Wavy

It is defined with sine function which
is inherently non-linear.

Accuracy 12 Layers single qubit:
93.6%

Iris versicolor vs virginica:

This classification has slight
overlap.

Accuracy 4 Layers single qubit:
93.8%

Member collaboration

Melissa: Developed the synthetic dataset generation functions, contributed to the implementation and evaluation of single-qubit and 3-qubit quantum classifiers (with and without entanglement), executed simulation runs including extended-depth experiments (8, 12, and 16 layers), consolidated and analyzed results, and supported code refactoring, did the repository maintenance, and GitHub version control.

Eylül: Contributed to the Iris dataset preprocessing and data standardization steps, participated in the implementation and evaluation of the 3-qubit quantum classifier without entanglement for 1, 2, 4, and 8 layers, executed the comparative analysis of models with and without entanglement focusing on train and test accuracy metrics, final formatting and arrangement of the presentation slides.

Sahasra: Programmed classical implementation of XGB, SVM, Random Forest, Linear Regression and FFNN on the generative and Iris Dataset. Implemented 2 qubit no entanglement and 3 qubit entanglement for various layers (1, 2, 4, 8, 12 layers) and different datasets. Analyzed the results, compared quantum and classical model architectures.

Elena: Implementation of single-qubit quantum classifier code with support in the runs by the others. Implementation of 2-qubit entangled and 3 qubit no entangled quantum classifier code with support in the runs by the others. Implementation of the Iris dataset's study with quantum circuit.